

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل چهاردهم: Vector space classification

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

14.1 Document representations and measures of relatedness in vector spaces

14.2 Rocchio classification

14.3 k nearest neighbor

14.3.1 Time complexity and optimality of kNN

14.4 Linear versus nonlinear classifiers

14.5 Classification with more than two classes

14.6 The bias-variance tradeoff

14.7 Chapter 4 Summary

14 طبقه‌بندی فضای برداری: چگونه با بردارها، متون را درک کنیم؟

گوگل ترنسلیت و چالش طبقه‌بندی متون: چگونه زبان‌ها را از هم تشخیص دهیم؟

تصور کنید در تیم گوگل ترنسلیت کار می‌کنید و باید سیستمی طراحی کنید که متون‌ها را به زبان‌های مختلفی (انگلیسی، فرانسوی، آلمانی، اسپانیایی و...) طبقه‌بندی کند. چگونه می‌توانید با استفاده از بردارهای عددی، زبان متن را بدون نیاز به درک معنای آن تشخیص دهید؟

این دقیقاً همان چالشی است که مدل فضای برداری با آن روبرو است: چگونه می‌توانیم با استفاده از ویژگی‌های عددی (مانند وزن tf-idf) به جای وجود یا عدم وجود واژه‌ها، متون‌ها را از هم تشخیص دهیم؟

در فصل 13، با مدل بیز ساده کار کردیم که در آن هر سند به صورت دنباله‌ای از واژه‌ها یا بردار دوتایی نمایش داده می‌شد. در این فصل، از نمایش متفاوتی استفاده می‌کنیم: **مدل فضای برداری** که در فصل 6 توسعه داده شد. در این مدل، هر سند به صورت یک بردار با یک مولفه حقیقی برای هر واژه (معمولاً وزن tf-idf) نمایش داده می‌شود. بنابراین، فضای سند X ، دامنه تابع طبقه‌بندی γ ، به صورت $\mathbb{R}^V$ است.

🔴 فرضیه تداوم (Contiguity hypothesis): مرزهای نامرئی بین زبان‌ها

فرضیه اصلی در استفاده از مدل فضای برداری، **فرضیه تداوم** است: اسناد یک کلاس ناحیه‌ای پیوسته تشکیل می‌دهند و مناطق کلاس‌های مختلف با هم همپوشانی ندارند.

در بسیاری از وظایف طبقه‌بندی متن، به خصوص نوعی که در فصل 13 با آن مواجه شدیم، کلاس‌ها را می‌توان با الگوهای واژگانی متمایز کرد. برای مثال، اسناد کلاس China معمولاً مقادیر بالایی در ابعاد مربوط به Chinese، Beijing و Mao دارند، در حالی که اسناد کلاس UK مقادیر بالایی برای London، British و Queen دارند. بنابراین، اسناد دو کلاس مناطق مجزایی را تشکیل می‌دهند که در شکل 14.1 نشان داده شده است و می‌توانیم مرزهایی را برای جداسازی آن‌ها ترسیم کرد.

مثال واقعی: در سیستم طبقه‌بندی مقالات علمی آرکایو (ArXiv)، مقالات مربوط به هوش مصنوعی معمولاً واژه‌هایی مانند "deep learning"، "neural network" و "algorithm" دارند، در حالی که مقالات مربوط به فیزیک نظری واژه‌هایی مانند "particle"، "quantum" و "field" دارند. این دو مجموعه در فضای برداری مناطق مجزایی را تشکیل می‌دهند که با مرزهای خطی قابل تفکیک هستند.

🔵 اهمیت نمایش سند: چرا کلمات توقف مهم هستند؟

اینکه آیا یک مجموعه از اسناد به مناطق مجزایی نگاشته می‌شود یا نه، به انتخاب‌هایی که برای نمایش سند انجام می‌دهیم بستگی دارد. برای مثال، وقوع مکرر ضمیر اول شخص "I" شواهدی برای کلاس "نوشته توسط یک نفر" است. اما این اطلاعات ممکن است از نمایش سند حذف شود اگر از لیست کلمات توقف استفاده کنیم.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان آمازون، کلماتی مانند "and"، "is"، "the" و "a" تقریباً در همه نظرات وجود دارند اما اطلاعات کمی در مورد دسته محصول ارائه می‌دهند. حذف این کلمات به سیستم کمک می‌کند تا روی کلمات کلیدی مانند "poor"، "excellent" یا "packaging" تمرکز شود که اطلاعات بیشتری در مورد نظر مشتری ارائه می‌دهند.

🔴 روش‌های طبقه‌بندی فضای برداری

این فصل دو روش طبقه‌بندی فضای برداری را معرفی می‌کند:

- **طبقه‌بندی Rocchio** (بخش 14.2): فضای برداری را به مناطقی تقسیم می‌کند که در مرکز مراکز یا نمونه‌های اولیه قرار دارند، یک برای هر کلاس که به صورت مرکز جرم تمام اسناد آن کلاس محاسبه

می‌شود. طبقه‌بندی Rocchio ساده و کارآمد است، اما اگر کلاس‌ها تقریباً کروی با شعاع‌های مشابه نباشند، دقیق نیست.

- **طبقه‌بندی kNN** (بخش 14.3): به سند آزمون، کلاس اکثریت از k نزدیک‌ترین همسایه‌ها را اختصاص می‌دهد. kNN نیازی به آموزش صریح ندارد و می‌تواند مستقیماً از مجموعه آموزشی استفاده کند. اگر مجموعه آموزشی بزرگ باشد، kNN می‌تواند کلاس‌های غیرکروی و پیچیده‌تر را بهتر از Rocchio مدیریت کند.

مثال واقعی: در سیستم طبقه‌بندی ایمیل‌های گوگل، Rocchio برای فیلتر اسپم‌های واضح (مانند ایمیل‌های تبلیغاتی که الگوی مشخصی دارند) استفاده می‌شود، در حالی که kNN برای فیلتر اسپم‌های پیچیده‌تر که الگوهای پنهان دارند یا از تکنیک‌های جدیدی استفاده می‌کنند، بهتر عمل می‌کند.

📈 طبقه‌بندهای خطی: مرزهای مستقیم در فضای برداری

تعداد زیادی از طبقه‌بندهای متن را می‌توان به عنوان **طبقه‌بندهای خطی** در نظر گرفت - طبقه‌بندهایی که بر اساس ترکیب خطی ساده‌ای از ویژگی‌ها طبقه‌بندی می‌کنند. چنین طبقه‌بندهایی فضای ویژگی‌ها را به مناطقی جدا شده توسط هایپرپلن‌های تصمیم خطی تقسیم می‌کنند.

وقتی از طبقه‌بندهای دو کلاسی برای مسائل با بیش از دو کلاس استفاده می‌کنیم، دو نوع وظیفه وجود دارد:

- **وظایف یک-از-چند (one-of):** سند باید دقیقاً به یکی از چندین کلاس انحصاری متقابل اختصاص داده شود.

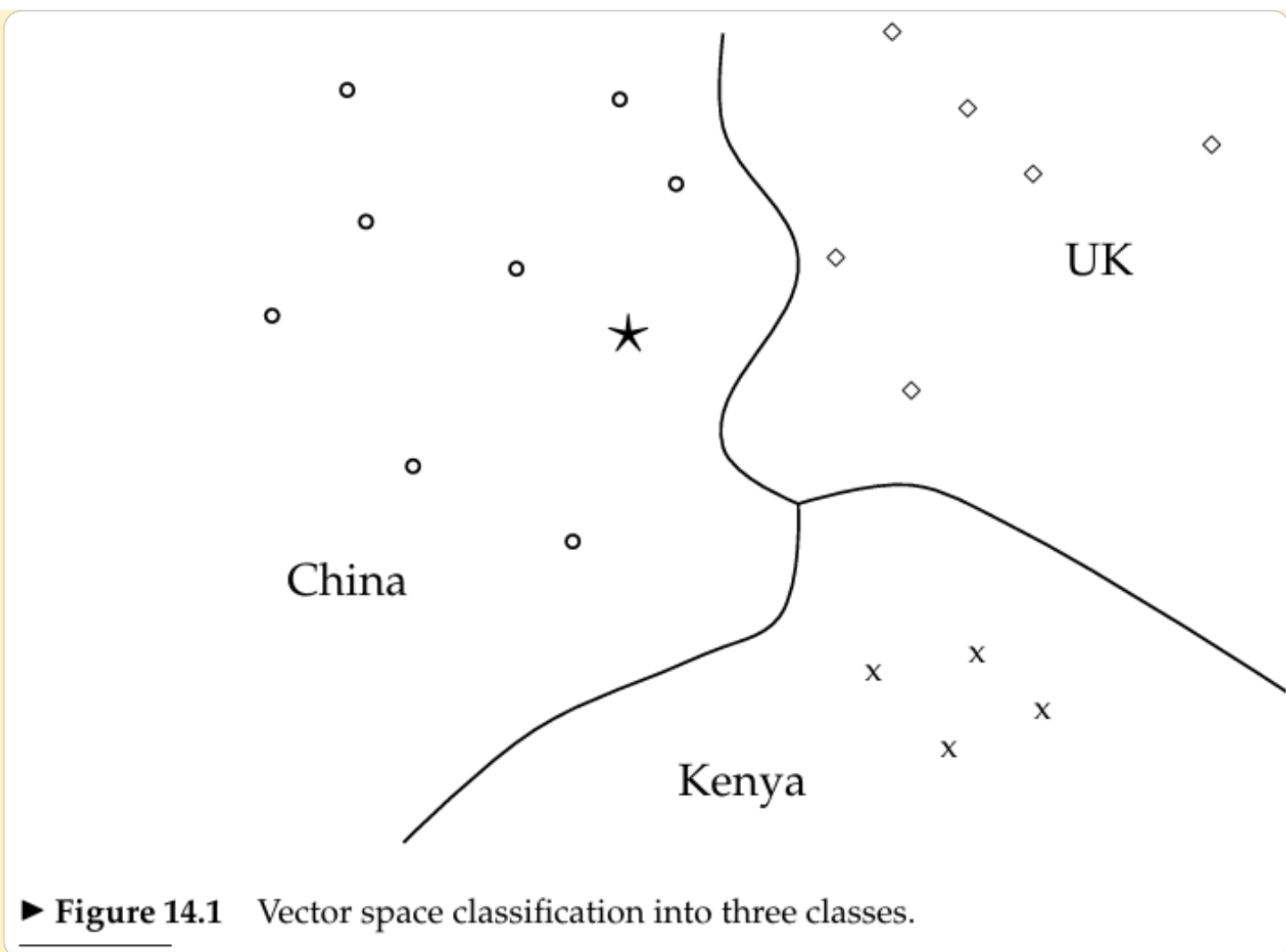
- **وظایف هر-از-چند (any-of):** سند می‌تواند به هر تعدادی از کلاس‌ها اختصاص داده شود (در بخش 14.5 توضیح داده خواهد شد).

طبقه‌بندهای دو کلاسی وظایف هر-از-چند را حل می‌کنند و می‌توانند برای حل وظایف یک-از-چند ترکیب شوند.

14.1 نمایش اسناد و معیارهای وابستگی در فضاهای برداری 📊

💡 بردارها به صورت نقطاطی: از ابعاد به مرزها

برای نمایش ویژگی‌های بردار اسناد در طبقه‌بندی فضای برداری، این بردارها را به صورت نقاطی در یک صفحه نمایش می‌دهیم مانند مثال در شکل 14.1. در واقعیت، بردارهای سند بردارهای یکانی نرمال‌شده هستند که به سطح یک ابرکره (hypersphere) اشاره می‌کنند.



شکل 14.2 تصویر ابعاد کوچک از سطح یک ابرکره را به یک صفحه نشان می‌دهد. سمت چپ: تصویر دایره‌ای دو بعدی به یک بعدی. برای نقاط x_1, x_2, x_3, x_4, x_5 در مختصات x ، فاصله $|x_2 x_3|$ تقریباً 0.201 است و تنها 0.5% با $|x|$ تفاوت دارد. سمت راست: تصویر مربوط به نیمکره سه بعدی به دو بعدی.

فاصله‌ها و شباهت‌ها: چگونه "نزدیکی" را اندازه‌گیری کنیم؟

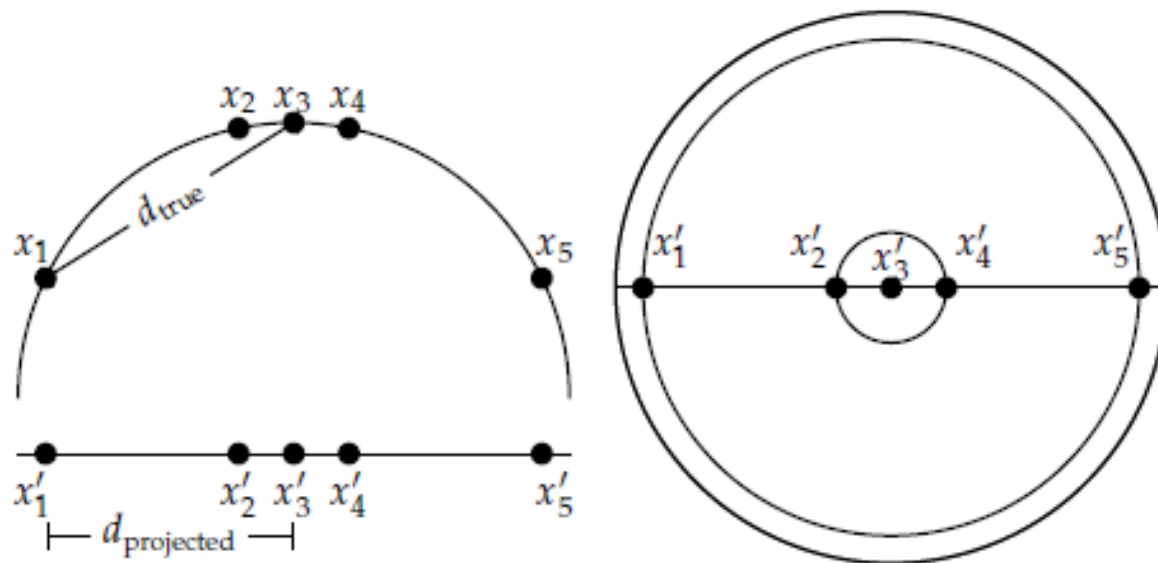
تصمیمات بسیاری از طبقه‌بندی‌های فضای برداری بر اساس مفهوم **فاصله** است، مثلاً هنگام محاسبه نزدیک‌ترین همسایه‌ها در طبقه‌بندی kNN. در این فصل، از فاصله اقلیدسی (Euclidean distance) به عنوان معیار فاصله اصلی استفاده می‌کنیم.

همانطور که قبلاً مشاهده شد (تمرین 6.18، صفحه 131 کتاب)، ارتباط مستقیمی بین شباهت کسینوسی (cosine similarity) و فاصله اقلیدسی برای بردارهای نرمال‌شده وجود دارد. در طبقه‌بندی فضای برداری، به ندرست اهمیت ندارد که وابستگی دو سند به صورت شباهت یا فاصله بیان شود.

مراکزها و میانگین‌ها: نقاط مرجع در فضای برداری

علاوه بر اسناد، **مراکزها** یا میانگین بردارها نیز نقش مهمی در طبقه‌بندی فضای برداری ایفا می‌کنند. مراکزها نرمال‌شده نیستند. برای بردارهای نرمال‌شده، حاصل ضرب داخلی، شباهت کسینوسی و فاصله اقلیدسی رفتار متفاوتی در حالت کلی دارند.

مثال واقعی: در سیستم طبقه‌بندی مقالات علمی آرکایو (ArXiv)، مراکز هر حوزه (مانند یادگیری ماشین، فیزیک نظری، زیست‌شناسی) به صورت میانگین بردارهای تمام مقالات آن حوزه محاسبه می‌شود. وقتی مقاله جدیدی به سیستم داده می‌شود، با محاسبه فاصله آن از مراکزهای مختلف، می‌توان تشخیص داد که به کدام حوزه نزدیک‌تر است.



► **Figure 14.2** Projections of small areas of the unit sphere preserve distances. Left: A projection of the 2D semicircle to 1D. For the points x_1, x_2, x_3, x_4, x_5 at x coordinates $-0.9, -0.2, 0, 0.2, 0.9$ the distance $|x_2 x_3| \approx 0.201$ only differs by 0.5% from $|x'_2 x'_3| = 0.2$; but $|x_1 x_3|/|x'_1 x'_3| = d_{\text{true}}/d_{\text{projected}} \approx 1.06/0.9 \approx 1.18$ is an example of a large distortion (18%) when projecting a large area. Right: The corresponding projection of the 3D hemisphere to 2D.

شکل 14.2 تصویر ابعاد کوچک از سطح یک ابرکره را به یک صفحه نشان می‌دهد. سمت چپ: تصویر دایره‌ای دو بعدی به یک بعدی. برای نقاط x_1, x_2, x_3, x_4, x_5 در مختصات x ، فاصله $|x_2 x_3|$ تقریباً 0.201 است و تنها 0.5% با $|x'_2 x'_3|$ تفاوت دارد. سمت راست: تصویر مربوط به نیمکره سه بعدی به دو بعدی.

این تصویر نشان می‌دهد که چگونه تصویر کردن به یک صفحه می‌تواند باعث تغییر شکل شود. برای مثال، تصویر دایره‌ای دو بعدی به یک بعدی به دلیل انحنای آن، کشیده شده و به صورت بیضی در می‌آید. این همان پدیده‌ای است که در سیستم‌های طبقه‌بندی متون رخ می‌دهد: وقتی زبان‌های مختلف را به فضای برداری کاهش می‌دهیم، ممکن است اطلاعات مهمی از دست بدهیم.

تمرین 14.1: انحنای در تصویر کردن

برای مناطق کوچک، فاصله‌ها روی سطح ابرکره به خوبی توسط فاصله‌ها روی تصویر آن‌ها تقریب می‌شوند زیرا برای زوایای کوچک، $\alpha \approx \sin \alpha$. برای چه اندازه زاویه، تغییر شکل $\alpha/\sin(\alpha)$ است:

برای زوایای کوچک:

- (i) $\alpha/\sin(\alpha) \approx 1.01$ (%تغییر شکل حدود 1)
- (ii) $\alpha/\sin(\alpha) \approx 1.05$ (%تغییر شکل حدود 5)
- (iii) $\alpha/\sin(\alpha) \approx 1.1$ (%تغییر شکل حدود 10)

این تمرین نشان می‌دهد که حتی برای مناطق کوچک، تصویر کردن به یک صفحه کمتر باعث تغییر شکل قابل توجهی می‌شود. برای مثال، تصویر نیمکره سه بعدی به دو بعدی می‌تواند باعث تغییر شکل 18% شود (مثال سمت راست).

تفسیر عملی: در سیستم طبقه‌بندی متون گوگل، وقتی زبان‌های مختلف را به فضای برداری با ابعاد کمتر کاهش می‌دهیم (برای افزایش سرعت)، ممکن است برخی ویژگی‌های ظریف که زبان‌ها را از هم متمایز می‌کنند از دست

بدهیم. این می‌تواند باعث کاهش دقت طبقه‌بندی شود، به خصوص برای زبان‌هایی با ویژگی‌های ظریف که ساختار پیچیده‌تری دارند.

In [8]: *# نمایش اسناد به صورت بردار و محاسبه معیارهای وابستگی در فضاهاى بردارى*

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import normalize

def euclidean_distance(v1, v2):
    """
    محاسبه فاصله اقلیدسی بین دو بردار

    این تابع فاصله اقلیدسی استاندارد بین دو بردار را محاسبه می‌کند

    پارامترها:
    v1, v2: می‌خواهیم فاصله بین آن‌ها را محاسبه کنیم

    خروجی: مقدار فاصله اقلیدسی
    """
    return np.sqrt(np.sum((v1 - v2) ** 2))

def cosine_similarity(v1, v2):
    """
    محاسبه شباهت کسینوسی بین دو بردار

    این تابع شباهت کسینوسی را بین دو بردار محاسبه می‌کند

    پارامترها:
    v1, v2: می‌خواهیم شباهت بین آن‌ها را محاسبه کنیم

    خروجی: مقدار شباهت کسینوسی (بین 0 و 1)
    """
    dot_product = np.dot(v1, v2)
    norm_v1 = np.linalg.norm(v1)
    norm_v2 = np.linalg.norm(v2)

    if norm_v1 == 0 or norm_v2 == 0:
        return 0

    return dot_product / (norm_v1 * norm_v2)

def visualize_vectors_2d(vectors, labels, centroids=None):
    """
    نمایش بردارهای دو بعدی در یک صفحه

    این تابع بردارهای دو بعدی را به صورت نقاطی در یک صفحه نمایش می‌دهد
    . همچنین می‌تواند مراکزها را در صورت وجود نمایش دهد

    پارامترها:
    vectors: لیست بردارهای دو بعدی
    labels: لیست برچسب‌های مربوط به هر بردار
    centroids: لیست مراکزها (اختیاری) برای نمایش

    خروجی: نمایش بردارها و مراکزها در یک صفحه
    """
    plt.figure(figsize=(10, 8))

    # رسم بردارها
    colors = ['red', 'blue', 'green', 'purple', 'orange']
    for i, (vec, label) in enumerate(zip(vectors, labels)):
        plt.quiver(0, 0, vec[0], vec[1], angles='xy', scale_units='xy', scale=1,
                  color=colors[i % len(colors)], alpha=0.7, width=0.005)
        plt.scatter(vec[0], vec[1], s=100, c=colors[i % len(colors)], label=label)

    # رسم مراکزها در صورت وجود
    if centroids is not None:
        for i, (centroid, label) in enumerate(zip(centroids, ['Centroid 1', 'Centroid 2'])):
            plt.scatter(centroid[0], centroid[1], s=200, c='black', marker='x', label=label)

    plt.axhline(0, color='gray', linestyle='--', alpha=0.3)
    plt.axvline(0, color='gray', linestyle='--', alpha=0.3)
    plt.grid(True, alpha=0.3)
    plt.xlabel('first dimension')
    plt.ylabel('second dimension')
    plt.title('2D vector demonstration')
    plt.legend()
    plt.axis('equal')
    plt.show()

def angle_distortion(angle_deg):
    """
    برای یک زاویه  $\alpha/\sin(\alpha)$  محاسبه نسبت تغییر شکل

    را برای یک زاویه محاسبه می‌کند  $\alpha/\sin(\alpha)$  این تابع نسبت تغییر شکل
    . این معیار نشان می‌دهد که چقدر تصویر یک زاویه کوچک در یک صفحه تغییر شکل می‌دهد

    پارامترها:
    angle_deg: زاویه بر حسب درجه

    خروجی: مقدار  $\alpha/\sin(\alpha)$ 
    """
```

```
"""
angle_rad = np.radians(angle_deg)
return angle_rad / np.sin(angle_rad)

# تمرین 14.1: محاسبه تغییر شکل برای زوایای کوچک
print("تمرین 14.1: محاسبه تغییر شکل برای زوایای کوچک")
print("="*50)
angles = [1, 5, 10]
for angle in angles:
    distortion = angle_distortion(angle)
    print(f"α/sin(α) = {distortion:.4f}" زاویه {angle} نسبت تغییر شکل")

# ایجاد چند بردار نمونه برای نمایش
vectors = np.array([
    [2, 3], # کلاس 1
    [4, 1], # کلاس 2
    [1, 4], # کلاس 3
    [3, 2] # کلاس 4
])
labels = ["class 1", "class 2", "class 3", "class 4"]

# محاسبه و نمایش بردارها
print("\nنمایش بردارهای دو بعدی:")
visualize_vectors_2d(vectors, labels)

# محاسبه شباهت کسینوسی و فاصله اقلیدسی بین بردارها
print("\nمعیارهای وابستگی بین بردارها:")
for i in range(len(vectors)):
    for j in range(i+1, len(vectors)):
        cos_sim = cosine_similarity(vectors[i], vectors[j])
        euc_dist = euclidean_distance(vectors[i], vectors[j])
        print(f"{i+1} و {j+1}: {cos_sim:.4f}" شباهت کسینوسی بین بردار)
        print(f"{i+1} و {j+1}: {euc_dist:.4f}" فاصله اقلیدسی بین بردار)

# نرمال‌سازی بردارها و مقایسه مجدد
normalized_vectors = normalize(vectors, norm='l2')
print("\nپس از نرمال‌سازی:")
for i in range(len(normalized_vectors)):
    for j in range(i+1, len(normalized_vectors)):
        cos_sim = cosine_similarity(normalized_vectors[i], normalized_vectors[j])
        euc_dist = euclidean_distance(normalized_vectors[i], normalized_vectors[j])
        print(f"{i+1} و {j+1}: {cos_sim:.4f}" شباهت کسینوسی بین بردار نرمال)
        print(f"{i+1} و {j+1}: {euc_dist:.4f}" فاصله اقلیدسی بین بردار نرمال)

# نمایش تفاوت بین بردارهای نرمال‌شده و نرمال‌نشده
print("\nمقایسه بردارهای نرمال‌شده و نرمال‌نشده:")
plt.figure(figsize=(12, 5))
orig_vectors = np.array([[2, 3], [4, 1]]) # بردارهای اصلی
norm_vectors = normalize(orig_vectors, norm='l2') # بردارهای نرمال‌شده

# نمایش بردارهای اصلی
plt.subplot(1, 2, 1)
plt.quiver(0, 0, orig_vectors[0, 0], orig_vectors[0, 1], angles='xy', scale_units='xy', scale=1, color='red', alpha=0.7, width=0.005)
plt.quiver(0, 0, orig_vectors[1, 0], orig_vectors[1, 1], angles='xy', scale_units='xy', scale=1, color='blue', alpha=0.7, width=0.005)
plt.scatter(orig_vectors[:, 0], orig_vectors[:, 1], s=100, c=['red', 'blue'])
plt.title('Original vectors')
plt.axis('equal')
plt.grid(True, alpha=0.3)

# نمایش بردارهای نرمال‌شده
plt.subplot(1, 2, 2)
plt.quiver(0, 0, norm_vectors[0, 0], norm_vectors[0, 1], angles='xy', scale_units='xy', scale=1, color='red', alpha=0.7, width=0.005)
plt.quiver(0, 0, norm_vectors[1, 0], norm_vectors[1, 1], angles='xy', scale_units='xy', scale=1, color='blue', alpha=0.7, width=0.005)
plt.scatter(norm_vectors[:, 0], norm_vectors[:, 1], s=100, c=['red', 'blue'])
plt.title('Normalized vectors')
plt.axis('equal')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# نمایش تفاوت در معیارهای وابستگی
print("\nتفسیر عملی:")
print("="*50)
print("1. برای زوایای کوچک، تصویر کردن به یک صفحه کمتر باعث تغییر شکل قابل توجهی می‌شود.")
print("2. در سیستم طبقه‌بندی متون، وقتی زبان‌ها را به فضای برداری کاهش می‌دهیم، ممکن است برخی ویژگی‌های ظریف که زبان‌ها را از هم متمایز می‌کنند از دست بدهیم.")
print("3. این می‌تواند باعث کاهش دقت طبقه‌بندی شود، به خصوص برای زبان‌هایی با ویژگی‌های ظریف که ساختار پیچیده‌تری دارند.")
print("4. با این حال، سیستم‌هایی مانند گوگل ترنسلیت ممکن است بین سرعت و دقت تعادل برقرار کنند.")
```

تمرین 14.1: محاسبه تغییر شکل برای زوایای کوچک

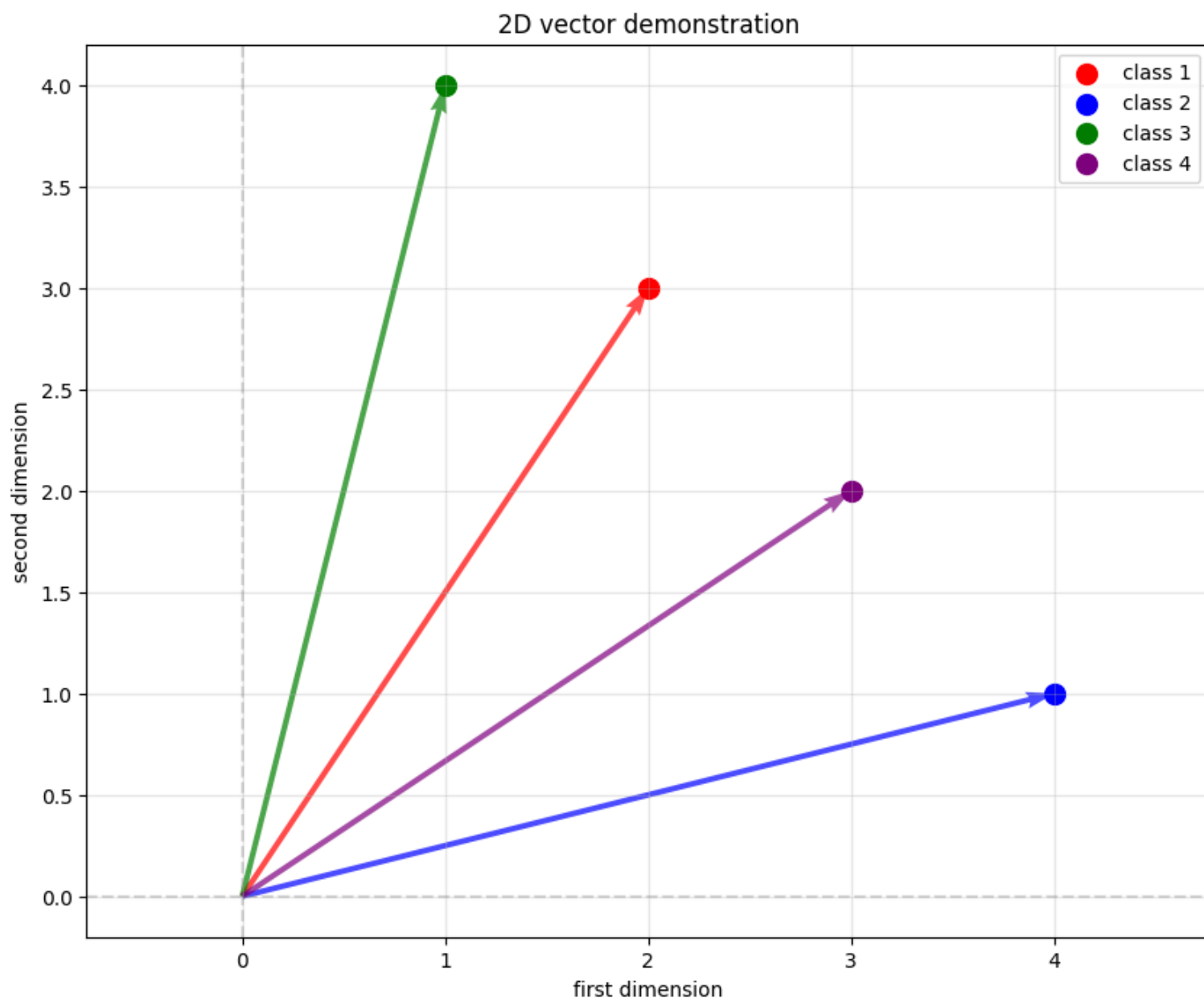
=====

$\alpha/\sin(\alpha)$ = 1.0001 زاویه 1 درجه: نسبت تغییر شکل

$\alpha/\sin(\alpha)$ = 1.0013 زاویه 5 درجه: نسبت تغییر شکل

$\alpha/\sin(\alpha)$ = 1.0051 زاویه 10 درجه: نسبت تغییر شکل

نمایش بردارهای دو بعدی:



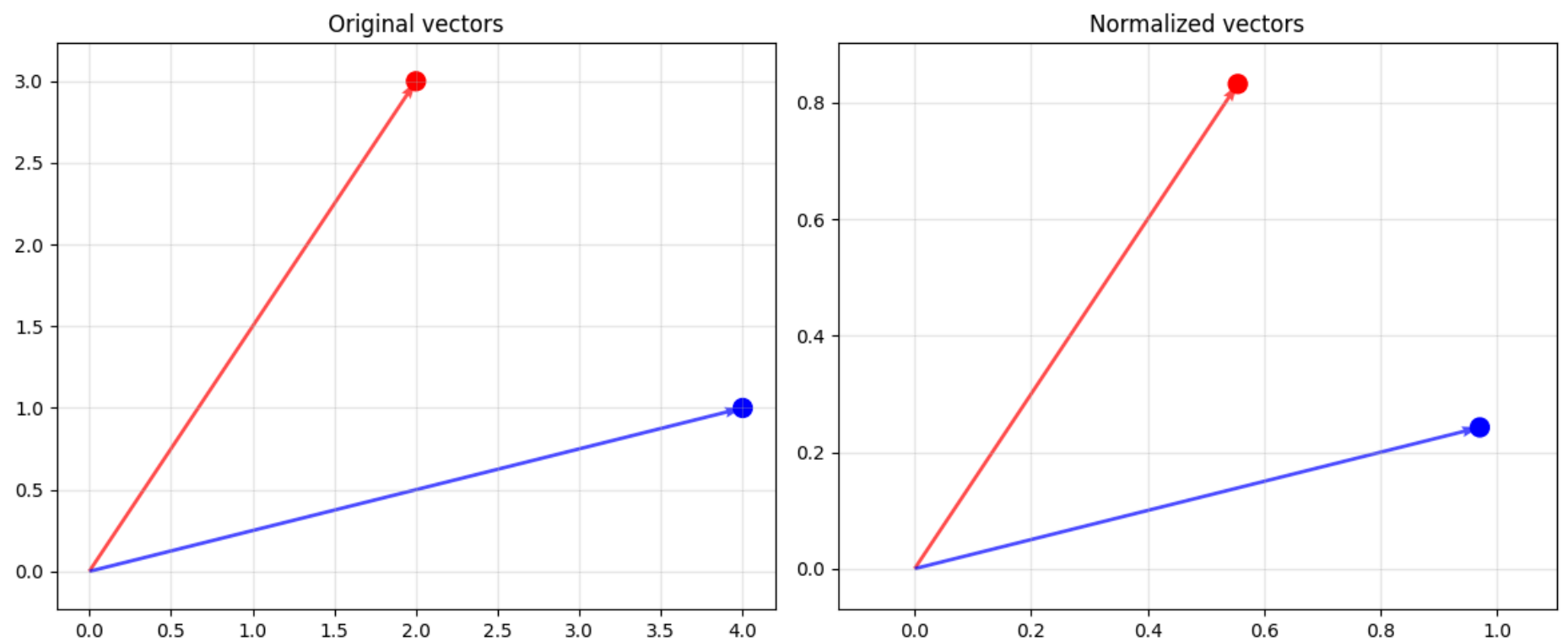
معیارهای وابستگی بین بردارها

شباهت کسینوسی بین بردار 1 و 2 : 0.7399
 فاصله اقلیدسی بین بردار 1 و 2 : 2.8284
 شباهت کسینوسی بین بردار 1 و 3 : 0.9417
 فاصله اقلیدسی بین بردار 1 و 3 : 1.4142
 شباهت کسینوسی بین بردار 1 و 4 : 0.9231
 فاصله اقلیدسی بین بردار 1 و 4 : 1.4142
 شباهت کسینوسی بین بردار 2 و 3 : 0.4706
 فاصله اقلیدسی بین بردار 2 و 3 : 4.2426
 شباهت کسینوسی بین بردار 2 و 4 : 0.9417
 فاصله اقلیدسی بین بردار 2 و 4 : 1.4142
 شباهت کسینوسی بین بردار 3 و 4 : 0.7399
 فاصله اقلیدسی بین بردار 3 و 4 : 2.8284

پس از نرمال‌سازی:

شباهت کسینوسی بین بردار نرمال 1 و 2 : 0.7399
 فاصله اقلیدسی بین بردار نرمال 1 و 2 : 0.7212
 شباهت کسینوسی بین بردار نرمال 1 و 3 : 0.9417
 فاصله اقلیدسی بین بردار نرمال 1 و 3 : 0.3413
 شباهت کسینوسی بین بردار نرمال 1 و 4 : 0.9231
 فاصله اقلیدسی بین بردار نرمال 1 و 4 : 0.3922
 شباهت کسینوسی بین بردار نرمال 2 و 3 : 0.4706
 فاصله اقلیدسی بین بردار نرمال 2 و 3 : 1.0290
 شباهت کسینوسی بین بردار نرمال 2 و 4 : 0.9417
 فاصله اقلیدسی بین بردار نرمال 2 و 4 : 0.3413
 شباهت کسینوسی بین بردار نرمال 3 و 4 : 0.7399
 فاصله اقلیدسی بین بردار نرمال 3 و 4 : 0.7212

مقایسه بردارهای نرمال‌شده و نرمال‌نشده



- =====
1. برای زوایای کوچک، تصویر کردن به یک صفحه کمتر باعث تغییر شکل قابل توجهی می‌شود.
 2. در سیستم طبقه‌بندی متون، وقتی زبان‌ها را به فضای برداری کاهش می‌دهیم، ممکن است برخی ویژگی‌های ظریف که زبان‌ها را از هم متمایز می‌کنند از دست بدهیم.
 3. این می‌تواند باعث کاهش دقت طبقه‌بندی شود، به خصوص برای زبان‌هایی با ویژگی‌های ظریف که ساختار پیچیده‌تری دارند.
 4. با این حال، سیستم‌هایی مانند گوگل ترنسلیت ممکن است بین سرعت و دقت تعادل برقرار کنند.

14.2 طبقه‌بندی Rocchio: مرزهای مستقیم در فضای برداری

🌐 **گوگل نیوز و چالش طبقه‌بندی اخبار: چگونه اخبار مرتبط با یک کشور را از اخبار دیگر جدا کنیم؟**

تصور کنید در تیم گوگل نیوز کار می‌کنید و باید سیستمی طراحی کند که اخبار مرتبط با کشورهای مختلف (چین، بریتانیا، آمریکا، ژاپن و...) را از هم جدا کند. این چالش پیچیده‌ای است زیرا اخبار مربوط به یک کشور ممکن است واژه‌های مشابهی با اخبار کشور دیگر داشته باشد.

طبقه‌بندی Rocchio یکی از بهترین روش‌ها برای محاسبه مرزهای تصمیم خوب در فضای برداری است. این روش از **مراکزها** یا **نمونه‌های اولیه** برای تعریف کلاس‌ها استفاده می‌کند و سپس بر اساس فاصله، سند را به کلاسی اختصاص می‌دهد که مراکز آن به سند نزدیک‌تر است.

در طبقه‌بندی Rocchio، مراکز کلاس c به صورت زیر محاسبه می‌شود:

$$\tilde{\mu}(c) = \frac{1}{|D_c|} \sum_{d \in D_c} \tilde{\mathbf{v}}(d)$$

که در آن D_c مجموعه اسنادی از کلاس c و $\tilde{\mathbf{v}}(d)$ بردار نرمال‌شده سند d است.

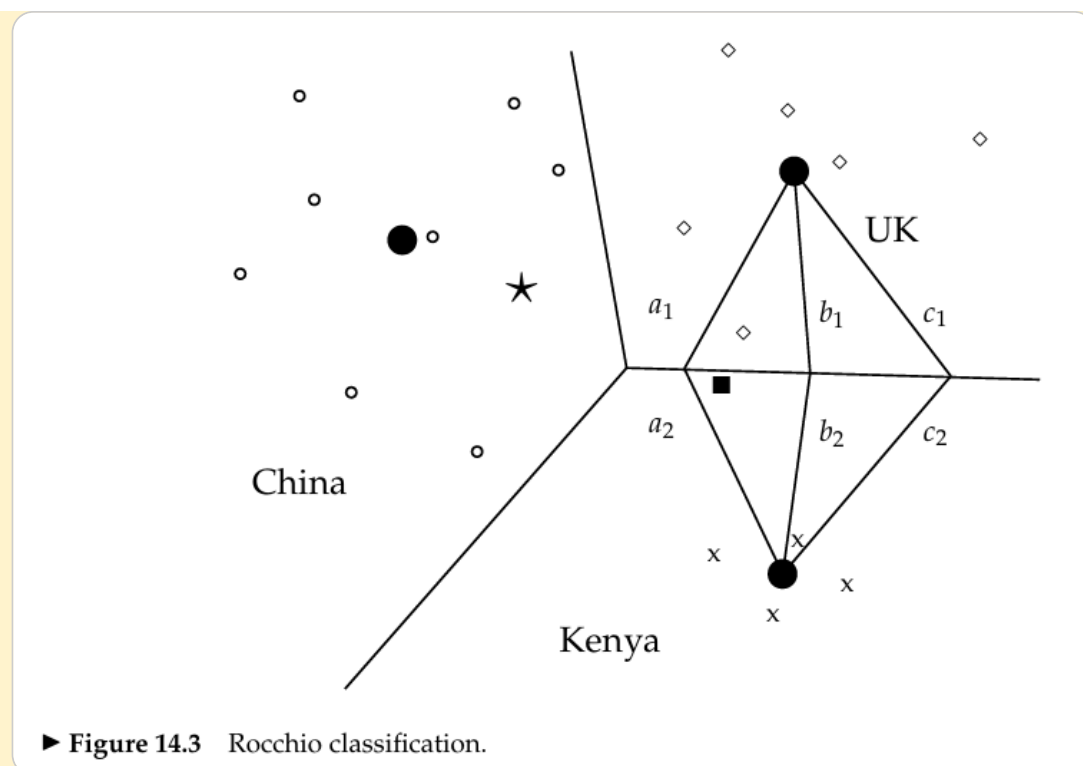
مرز بین دو کلاس در طبقه‌بندی Rocchio، مجموعه‌ای از نقاط است که فاصله آن‌ها از دو مراکز برابر است. در فضای دو بعدی، این مرز یک خط است. در فضای M بعدی، این مرز یک هایپرپلن است که به صورت زیر تعریف می‌شود:

$$\tilde{\mathbf{w}} \cdot \tilde{\mathbf{x}} = b$$

که در آن $\tilde{\mathbf{v}}(\mathbf{w})$ بردار نرمال‌شده هایپرپلن و b یک ثابت است. این تعریف شامل خطوط (هر خط در فضای 2D) و صفحات (هر صفحه در فضای 3D) و هایپرپلن‌ها در فضاهای با ابعاد بالاتر می‌شود.

📐 **شکل 14.3: مرزهای تصمیم Rocchio برای سه کلاس**

شکل 14.3 مرزهای تصمیم Rocchio را برای سه کلاس (UK، China و Kenya) نشان می‌دهد. اسناد به صورت دایره‌ها، الماس‌ها و نماد X نمایش داده شده‌اند. مرز بین دو کلاس مجموعه‌ای از نقاط است که فاصله آن‌ها از دو مراکز برابر است. برای مثال، مرز بین کلاس China و Kenya یک خط است که به صورت $|\tilde{\mathbf{v}}(d_1)| = |\tilde{\mathbf{v}}(d_2)|$ تعریف می‌شود.



قانون طبقه‌بندی در Rocchio به این صورت است که سند به کلاسی اختصاص داده می‌شود که مراکز آن به سند نزدیک‌تر است. به طور معادل، ما مراکزی را پیدا می‌کنیم که سند به آن نزدیک‌تر است و سپس سند را به کلاس مربوطه اختصاص می‌دهیم.

جدول 14.1 بردارها و مراکزها را برای داده‌های جدول 13.1 نشان می‌دهد. برای مثال، بردار سند $d_5(0, 0.71, 0.71, 0)$ مربوط به خبری است که حاوی واژه‌های "Tokyo"، "Japan"، "Chinese" و "Macao" است. مراکز کلاس China بر اساس این داده‌ها $\tilde{v}(\mu_{China}) = (0.33, 0.33, 0.33, 0)$ است.

	term weights					
vector	Chinese	Japan	Tokyo	Macao	Beijing	Shanghai
$\vec{d}_1$	0	0	0	0	1.0	0
$\vec{d}_2$	0	0	0	0	0	1.0
$\vec{d}_3$	0	0	0	1.0	0	0
$\vec{d}_4$	0	0.71	0.71	0	0	0
$\vec{d}_5$	0	0.71	0.71	0	0	0
$\vec{\mu}_c$	0	0	0	0.33	0.33	0.33
$\vec{\mu}_{\bar{c}}$	0	0.71	0.71	0	0	0

$$|\tilde{v}(d_5) - \tilde{v}(\mu_{\text{China}})| \approx 1.15$$

بنابراین، سند به کلاس China اختصاص داده می‌شود زیرا به مراکز China نزدیک‌تر است.

```

TRAINROCCHIO(C, ID)
1  for each  $c_j \in C$ 
2  do  $D_j \leftarrow \{d : \langle d, c_j \rangle \in ID\}$ 
3      $\bar{\mu}_j \leftarrow \frac{1}{|D_j|} \sum_{d \in D_j} \vec{v}(d)$ 
4  return  $\{\bar{\mu}_1, \dots, \bar{\mu}_J\}$ 

APPLYROCCHIO( $\{\bar{\mu}_1, \dots, \bar{\mu}_J\}, d$ )
1  return  $\arg \min_j |\bar{\mu}_j - \vec{v}(d)|$ 

```

► Figure 14.4 Rocchio classification: Training and testing.

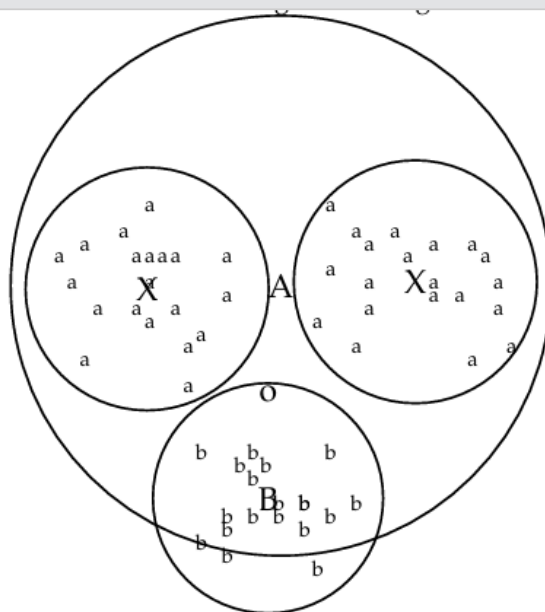
الگوریتم به این صورت عمل می‌کند که برای هر کلاس c_j :

1. تمام اسناد مربوط به آن کلاس (D_j) را جمع می‌کند.
 2. مراکز ($\bar{\mu}_j$) را به صورت میانگین بردارهای آن اسناد محاسبه می‌کند.
 3. برای طبقه‌بندی یک سند جدید d ، بردار آن $\vec{v}(d)$ را محاسبه کرده و به هر مراکز $\bar{\mu}_j$ فاصله را محاسبه می‌کند.
 4. سند را به کلاسی اختصاص می‌دهد که مراکز آن به سند نزدیک‌تر است.
- با این الگوریتم، سیستم می‌تواند تمام مراکزها و سپس قوانین طبقه‌بندی را برای طبقه‌بندی اسناد جدید محاسبه و ذخیره کند.

🎯 پیچیدگی‌های Rocchio: چه زمانی کار نمی‌کند؟

با وجود سادگی و کارایی بالا، طبقه‌بندی Rocchio محدودیت‌هایی دارد:

- **کلاس‌های چندوجهی (multimodal):** Rocchio فرض می‌کند که کلاس‌ها تقریباً کروی با شعاع‌های مشابه هستند. این فرض برای بسیاری از وظایف طبقه‌بندی متن درست نیست. مثال کلاسی "a" در شکل 14.5 شامل دو خوشه مجزا با شعاع‌های مشابه است.



► Figure 14.5 The multimodal class "a" consists of two different clusters (small upper circles centered on X's). Rocchio classification will misclassify "o" as "a" because it is closer to the centroid A of the "a" class than to the centroid B of the "b" class.

- **توزیع نابرابر:** Rocchio فقط از فاصله از مرکز استفاده می‌کند و جزئیات توزیع نقاط در هر کلاس را نادیده می‌گیرد. در شکل 14.3، Rocchio کلاس Kenya را به China اختصاص می‌دهد زیرا توزیع نقاط آن پراکندگی بیشتری دارد، در حالی که توزیع نقاط China فشرده‌تر است.

مثال واقعی: در سیستم طبقه‌بندی مقالات علمی آرکایو (ArXiv)، برخی حوزه‌ها مانند "یادگیری ماشین" ممکن است شامل زیرشاخه‌های متفاوتی (مانند یادگیری عمیق در مقابل یادگیری تقویتی) باشند که ساختار پیچیده‌ای دارند. طبقه‌بندی Rocchio ممکن است در تشخیص این زیرشاخه‌ها دچار شود و همه آن‌ها را به یک کلاس اصلی اختصاص دهد.

🔧 پیچیدگی زمانی: چگونه کارایی را محاسبه کنیم؟

جدول 14.2 زمان آموزش و آزمون Rocchio را با روش‌های دیگر مقایسه می‌کند. پیچیدگی زمانی Rocchio به صورت زیر محاسبه می‌شود:

$$\Theta(|D|L_{ave} + |C||V|)$$

mode	time complexity
training	$\Theta(D L_{ave} + C V)$
testing	$\Theta(L_a + C M_a) = \Theta(C M_a)$

► **Table 14.2** Training and test times for Rocchio classification. L_{ave} is the average number of tokens per document. L_a and M_a are the numbers of tokens and types, respectively, in the test document. Computing Euclidean distance between the class centroids and a document is $\Theta(|C|M_a)$.

که در آن $|D|$ تعداد اسناد، L_{ave} میانگین تعداد توکن در هر سند، $|C|$ تعداد کلاس‌ها و $|V|$ تعداد ویژگی‌ها است. این نشان می‌دهد که پیچیدگی زمانی Rocchio خطی در اندازه مجموعه آموزشی است.

در مقایسه، طبقه‌بندی Rocchio و بیز ساده هر دو پیچیدگی زمانی خطی یکسانی دارند، زیرا هر دو روش تنها نیاز به محاسبه مراکزها و سپس مقایسه فاصله‌ها دارد. این در حالی که هر دو روش به اندازه مجموعه آموزشی خطی است.

In [5]: # پیاده‌سازی طبقه‌بندی Rocchio

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import normalize

def train_rocchio(documents, labels):
    """
    Rocchio آموزش طبقه‌بند

    این تابع مراکزها را برای هر کلاس محاسبه کرده و برمی‌گرداند.
    """
    # tf-idf تبدیل اسناد به بردارهای
    vocab = set()
    for doc in documents:
        vocab.update(doc)

    vocab_list = list(vocab)
    # برای هر سند و واژه tf محاسبه
    tf_matrix = np.zeros((len(documents), len(vocab_list)))
    for i, doc in enumerate(documents):
        for j, term in enumerate(vocab_list):
            tf_matrix[i, j] = doc.count(term)

    # برای هر واژه df محاسبه
```

```

df = np.zeros(len(vocab_list))
for j, term in enumerate(vocab_list):
    df[j] = sum(1 for doc in documents if term in doc)

# محاسبه idf
n_docs = len(documents)
idf = np.log(n_docs / (df + 1))

# محاسبه tf-idf
tfidf_matrix = tf_matrix * idf

# نرمال‌سازی بردارها
normalized_vectors = normalize(tfidf_matrix, norm='l2')

# محاسبه مراکز برای هر کلاس
classes = list(set(labels))
centroids = {}

for c in classes:
    # شاخص‌های مربوط به کلاس c
    class_indices = [i for i, label in enumerate(labels) if label == c]
    class_vectors = normalized_vectors[class_indices]

    # محاسبه مراکز به صورت میانگین بردارهای آن کلاس
    centroids[c] = np.mean(class_vectors, axis=0)

# برای استفاده در مرحله آزمون idf بازگرداندن مراکزها، واژگان، و مقادیر
return centroids, vocab_list, idf, normalized_vectors

def apply_rocchio(centroids, vocab_list, idf, test_doc):
    """
    روی یک سند آزمون Rocchio اعمال طبقه‌بند
    این تابع سند آزمون را به کلاسی اختصاص می‌دهد که مراکز آن به سند نزدیکتر است
    """
    # tf-idf تبدیل سند آزمون به بردار
    test_vector = np.zeros(len(vocab_list))
    for j, term in enumerate(vocab_list):
        test_vector[j] = test_doc.count(term)

    # از مرحله آموزش idf با استفاده از tf-idf محاسبه
    tfidf_vector = test_vector * idf

    # نرمال‌سازی بردار آزمون
    normalized_test_vector = normalize(tfidf_vector.reshape(1, -1), norm='l2')

    # محاسبه فاصله از هر مراکز
    min_distance = float('inf')
    predicted_class = None

    for c, centroid in centroids.items():
        distance = np.linalg.norm(normalized_test_vector - centroid)
        if distance < min_distance:
            min_distance = distance
            predicted_class = c

    return predicted_class, normalized_test_vector

# مثال کاربردی: طبقه‌بندی اخبار مربوط به کشورها
documents = [
    ["Chinese", "Beijing", "Mao"],
    ["Chinese", "Shanghai", "Mao"],
    ["Chinese", "Taiwan", "Mao"],
    ["Chinese", "Tokyo", "Mao"],
    ["British", "London", "Queen"],
    ["British", "London", "Queen"],
    ["Kenyan", "Nairobi", "tourism"],
    ["Kenyan", "Nairobi", "safari"]
]
labels = ["China", "China", "China", "China", "UK", "UK", "Kenya", "Kenya"]

# Rocchio آموزش طبقه‌بند
centroids, vocab, idf_values, normalized_vectors = train_rocchio(documents, labels)

# نمایش مراکزها
print("مراکزهای محاسبه شده برای هر کلاس:")
for c, centroid in centroids.items():
    print(f"کلاس {c}: {centroid}")

# اعمال روی سند آزمون
test_doc = ["Chinese", "Beijing", "economy"]
predicted_class, normalized_test_vector = apply_rocchio(centroids, vocab, idf_values, test_doc)
print(f"\nsند آزمون: {test_doc}")
print(f"کلاس پیش‌بینی شده: {predicted_class}")

# نمایش فضای برداری و مرزهای تصمیم
plt.figure(figsize=(12, 8))
colors = {'China': 'red', 'UK': 'blue', 'Kenya': 'green'}

# نمایش اسناد آموزشی
for i, label in enumerate(labels):
    plt.scatter(normalized_vectors[i, 0], normalized_vectors[i, 1],
                c=colors[label], alpha=0.7, label=label)

# نمایش مراکزها
for c, centroid in centroids.items():
    plt.scatter(centroid[0], centroid[1], s=200, c='black', marker='x',

```

```
label=f"Centroid {c}")

# نمایش سند آزمون
plt.scatter(normalized_test_vector[0, 0], normalized_test_vector[0, 1],
            c='purple', marker='*', s=200, label="Test document")

# رسم مرزهای تصمیم (فقط بین دو کلاس اول)
centroid_china_2d = np.array([centroids["China"][0], centroids["China"][1]])
centroid_uk_2d = np.array([centroids["UK"][0], centroids["UK"][1]])

# محاسبه نقطه میانی خط
midpoint = (centroid_china_2d + centroid_uk_2d) / 2
# محاسبه بردار جهت
direction = centroid_uk_2d - centroid_china_2d
# محاسبه بردار عمود بر خط
normal = np.array([-direction[1], direction[0]])
normal = normal / np.linalg.norm(normal)

# رسم بخشی از خط
line_length = 0.8
line_start = midpoint - normal * line_length / 2
line_end = midpoint + normal * line_length / 2

plt.plot([line_start[0], line_end[0]], [line_start[1], line_end[1]],
        '--', alpha=0.5, color='gray')

plt.xlabel('first dimension')
plt.ylabel('second dimension')
plt.title('Rocchio classification for news articles')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.show()

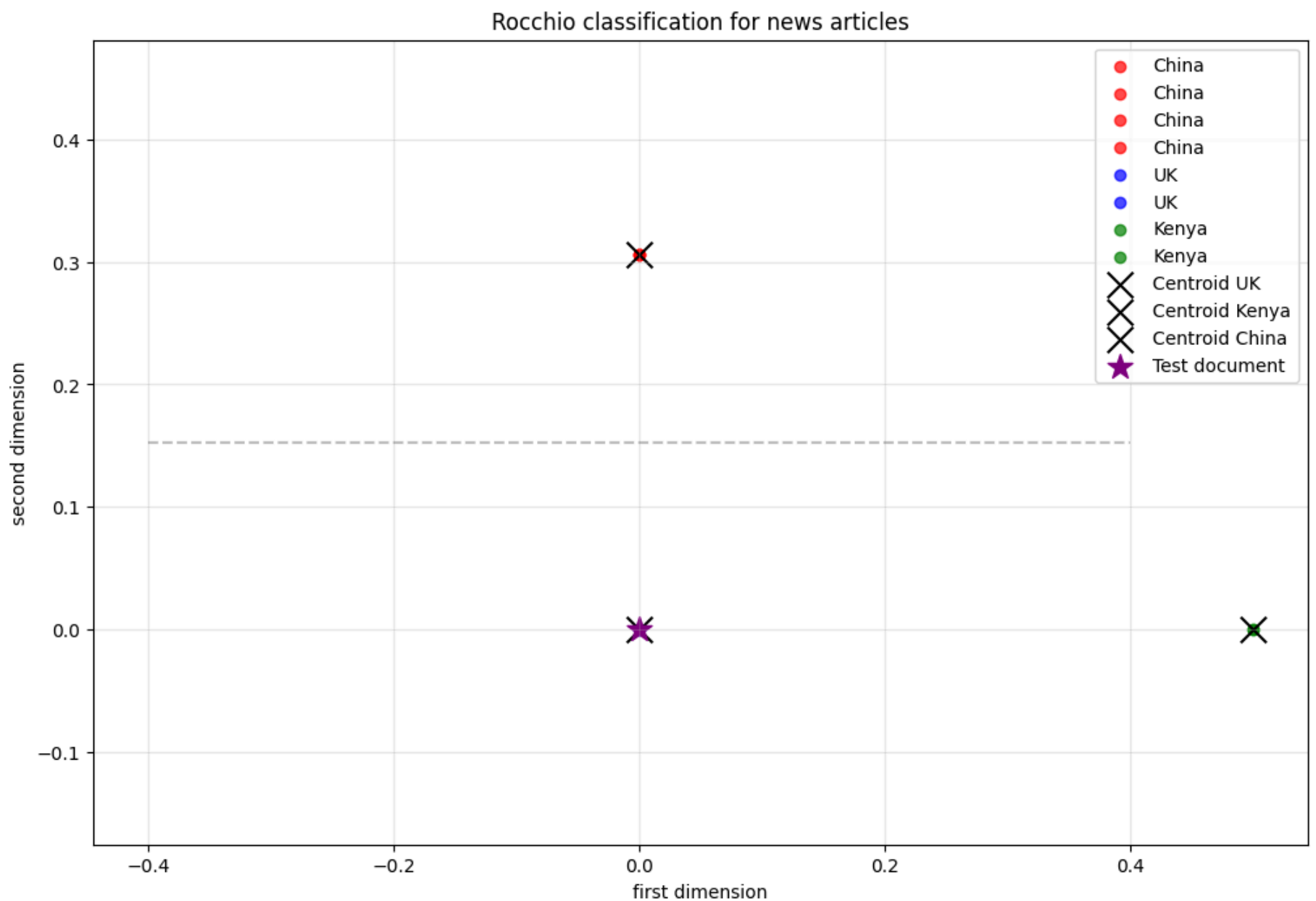
# کلاس‌های چندوجهی Rocchio: نمایش پیچیدگی
print("\nپیچیدگی‌های Rocchio:")
print("1. فرض می‌کند که کلاس‌ها تقریباً کروی با شعاع‌های مشابه هستند (multimodal): Rocchio")
print("2. فقط از فاصله از مرکز استفاده می‌کند و جزئیات توزیع نقاط در هر کلاس را نادیده می‌گیرد: Rocchio توزیع نابرابر")
print("3. مثال واقعی: در سیستم طبقه‌بندی مقالات علمی، برخی حوزه‌ها ممکن است شامل زیرشاخه‌های متفاوتی (مانند یادگیری ماشین) باشند که ساختار پیچیده‌ای دارند.")

# نمایش پیچیدگی زمانی
print("\nپیچیدگی زمانی:")
print("به صورت زیر محاسبه می‌شود Rocchio پیچیدگی زمانی:")
print("O(|D|Lave + |C||V|)")
print("تعداد ویژگی‌ها است |V| تعداد کلاس‌ها و |C| میانگین تعداد توکن در هر سند Lave، تعداد اسناد |D| که در آن")
print("خطی در اندازه مجموعه آموزشی است Rocchio این نشان می‌دهد که پیچیدگی زمانی")
print("و بیز ساده هر دو پیچیدگی زمانی خطی یکسانی دارند Rocchio در مقایسه، طبقه‌بندی")
```

مراکزهای محاسبه شده برای هر کلاس:

UK کلاس:	[0.0.577350270.0.0.57735027]
Kenya کلاس:	[0.500145590.0.0.353450410.500145590.0.0.35345041]
China کلاس:	[0.0.3057120.225427420.225427420.0.225427420.3057120.225427420.0.22542742]

سند آزمون: ['Chinese', 'Beijing', 'economy']
کلاس پیش‌بینی شده: China



پچیدگی‌های Rocchio:

- فرض می‌کند که کلاس‌ها تقریباً کروی با شعاع‌های مشابه هستند Rocchio (multimodal): کلاس‌های چندوجهی.
- فقط از فاصله از مرکز استفاده می‌کند و جزئیات توزیع نقاط در هر کلاس را نادیده می‌گیرد Rocchio: توزیع نابر.
- مثال واقعی: در سیستم طبقه‌بندی مقالات علمی، برخی حوزه‌ها ممکن است شامل زیرشاخه‌های متفاوتی (مانند یادگیری ماشین) باشند که ساختار پیچیده‌ای دارند.

پچیدگی زمانی:

به صورت زیر محاسبه می‌شود Rocchio پچیدگی زمانی:

$$\theta(|D|Lave + |C||V|)$$

تعداد ویژگی‌ها است $|V|$ تعداد کلاس‌ها و $|C|$ ، میانگین تعداد توکن در هر سند $Lave$ ، تعداد اسناد $|D|$ که در آن

خطی در اندازه مجموعه آموزشی است Rocchio این نشان می‌دهد که پچیدگی زمانی

و بیز ساده هر دو پچیدگی زمانی خطی یکسانی دارند Rocchio در مقایسه، طبقه‌بندی

14.3 طبقه‌بندی k نزدیک‌ترین همسایه (kNN): مرزهای پیچیده با جستجوی

محلی

اسپاتیفای و چالش جستجوی تصاویر: چگونه تصاویر مشابه را پیدا کنیم؟

تصور کنید در تیم اسپاتیفای (Spotify) کار می‌کنید و باید سیستمی طراحی کنید که موسیقی مشابه را بر اساس ویژگی‌های تصویری (رنگ، ریتم، ساز، ...) پیدا کند. این دقیقاً همان چالشی است که طبقه‌بندی k نزدیک‌ترین همسایه (kNN) با آن روبرو است: چگونه می‌توانیم با استفاده از شباهت بین بردارها، اسناد مشابه را پیدا کنیم؟

برخلاف Rocchio که از مراکزها استفاده می‌کند، kNN به طور مستقیم از مجموعه آموزشی استفاده می‌کند و نیازی به مرحله آموزش صریح ندارد. این روش به خصوص برای مسائلی که داده‌های جدید به طور مداوم در حال تغییر هستند، بسیار مفید است.

در طبقه‌بندی k نزدیک‌ترین همسایه (kNN)، هر سند آزمون به کلاسی اختصاص داده می‌شود که اکثریت از k نزدیک‌ترین همسایه‌های آن در مجموعه آموزشی به آن تعلق دارند. به طور معادل، ما k نزدیک‌ترین همسایه‌ها را پیدا می‌کنیم و سپس سند را به کلاس اکثریت اختصاص می‌دهیم.

```

TRAIN-KNN( $\mathcal{C}, \mathcal{D}$ )
1  $\mathcal{D}' \leftarrow \text{PREPROCESS}(\mathcal{D})$ 
2  $k \leftarrow \text{SELECT-K}(\mathcal{C}, \mathcal{D}')$ 
3 return  $\mathcal{D}', k$ 

```

```

APPLY-KNN( $\mathcal{C}, \mathcal{D}', k, d$ )
1  $S_k \leftarrow \text{COMPUTENEARESTNEIGHBORS}(\mathcal{D}', k, d)$ 
2 for each  $c_j \in \mathcal{C}$ 
3 do  $p_j \leftarrow |S_k \cap c_j|/k$ 
4 return  $\arg \max_j p_j$ 

```

► Figure 14.7 kNN training (with preprocessing) and testing. p_j is an estimate for $P(c_j|S_k) = P(c_j|d)$. c_j denotes the set of all documents in the class c_j .

الگوریتم kNN بر اساس فرضیه تداوم است که اسناد نزدیک به هم در فضای برداری به هم تعلق دارند. این روش به طور مستقیم از مجموعه آموزشی استفاده می‌کند و نیازی به مرحله آموزش صریح ندارد.

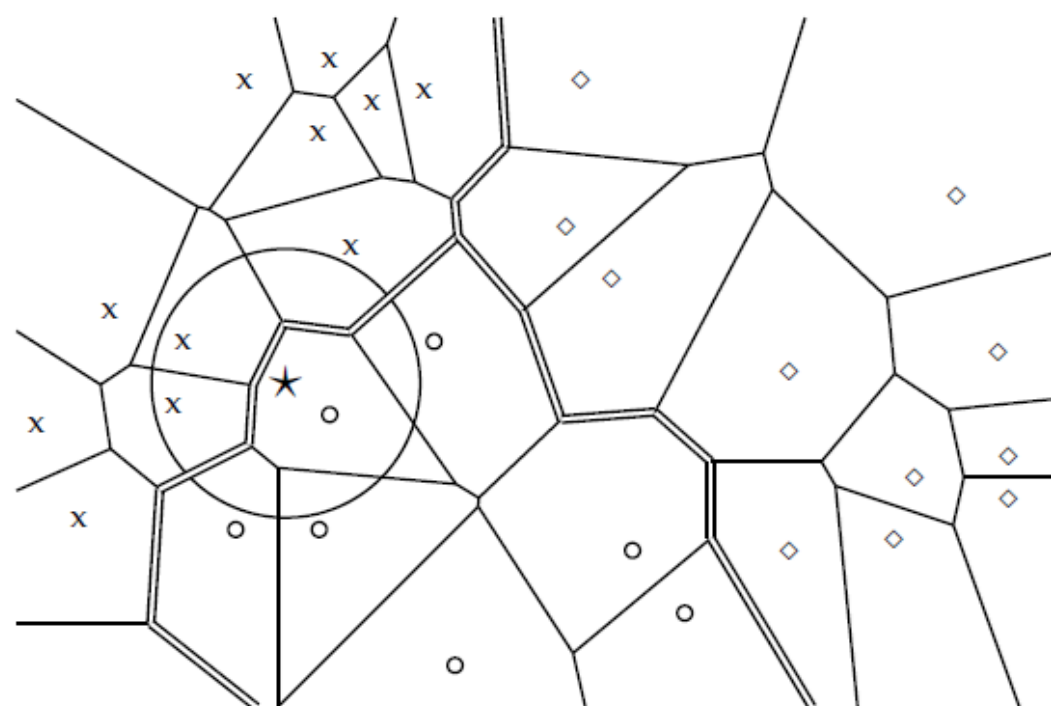
الگوریتم kNN به این صورت عمل می‌کند که برای هر سند آزمون d ، k نزدیک‌ترین همسایه‌های آن در مجموعه آموزشی را پیدا می‌کند:

1. فاصله هر سند آزمون d از هر کلاس c محاسبه می‌شود.

2. سند به کلاسی اختصاص داده می‌شود که دارای کمترین فاصله.

مثال واقعی: در سیستم اسپاتیفای، kNN برای پیدا کردن موسیقی مشابه بر اساس ویژگی‌های صوتی بسیار مفید است. این سیستم می‌تواند دو آهنگ با ریتم‌های متفاوت را به عنوان "مشابه" شناسایی کند، حتی اگر متون آن‌ها کاملاً متفاوت باشند. این کار به ویژگی‌های صوتی که مستقل از زبان و فرهنگ موسیقی هستند تکیه می‌دهد.

شکل 14.6: تقسیم فضای برداری به موزاییک‌های ورونوی



► Figure 14.6 Voronoi tessellation and decision boundaries (double lines) in 1NN classification. The three classes are: X, circle and diamond.

شکل 14.6 تقسیم فضای برداری را به موزاییک‌های ورونویی (Voronoi tessellation) تقسیم می‌کند. در این تقسیم، هر سند به یک سلول اختصاص داده می‌شود و هر سلول شامل اسنادی است که به آن کلاس تعلق دارند. مرزهای بین سلول‌ها هاینرپلن‌ها هستند که فضای بین سلول‌ها را به مناطقی تقسیم می‌کنند.

در M بعدی، هاینرپلن‌ها (M-1) فضای بین سلول‌ها را تقسیم می‌کنند که به صورت چندوجهی (M-1) بعدی تقسیم می‌شوند. این تقسیم‌ها می‌توانند ساختارهای پیچیده‌ای مانند کره‌ها، دونات و اشکال را نمایش دهند.

🔗 پیچیدگی‌های kNN: چه زمانی کار نمی‌کند؟

با وجود سادگی و کارایی بالا، طبقه‌بندی kNN محدودیت‌هایی دارد:

- **حساسیت بالا در مجموعه‌های بزرگ:** برای مجموعه‌های بزرگ، محاسبه فاصله برای هر سند آزمون ممکن است زمان‌بر باشد.
- **حساسیت ذخیره:** kNN به حافظه نیاز دارد تا تمام اسناد آموزشی را ذخیره کند.
- **عدم تعادل وزن‌دهی ویژگی‌ها:** ویژگی‌هایی با مقادیر بزرگ ممکن است بر ویژگی‌های با مقادیر کوچک تأثیر بیشتری داشته باشند.

مثال واقعی: در سیستم طبقه‌بندی تصاویر پینترست (Pinterest)، kNN برای پیدا کردن تصاویر مشابه بر اساس ویژگی‌های بصری ممکن است بسیار کارآمد باشد. با این حال، سیستم می‌تواند تصاویر مشابه را حتی اگر برچسب‌ها یا توضیحات متفاوتی داشته باشند پیدا کند.

⚙️ پیچیدگی زمانی: چگونه کارایی را بهبود آوریم؟

پیچیدگی زمانی kNN به صورت زیر محاسبه می‌شود:

$$\mathcal{O}(|D|L_{ave} + |C||V|)$$

که در آن |D| تعداد اسناد، L_{ave} میانگین تعداد توکن در هر سند، |C| تعداد کلاس‌ها و |V| تعداد ویژگی‌ها است. این نشان می‌دهد که پیچیدگی زمانی kNN خطی در اندازه مجموعه آموزشی است.

در مقایسه، پیچیدگی زمانی kNN معمولاً از Rocchio بدتر است زیرا نیاز به محاسبه و ذخیره تمام اسناد آموزشی دارد. با این حال، پیچیدگی زمانی Rocchio خطی در اندازه مجموعه آموزشی است، زیرا هر دو روش تنها نیاز به محاسبه مراکزها و سپس مقایسه فاصله‌ها دارد.

In [9]: `# نزدیک‌ترین همسایه k پیاده‌سازی طبقه‌بندی`

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.decomposition import PCA

def euclidean_distance(v1, v2):
    """
    محاسبه فاصله اقلیدسی بین دو بردار

    این تابع فاصله اقلیدسی استاندارد بین دو بردار را محاسبه می‌کند.

    پارامترها
```

.دو بردار که می‌خواهیم فاصله بین آن‌ها را محاسبه کنیم: v_1 , v_2 .

خروجی: مقدار فاصله اقلیدسی
"""

```
return np.sqrt(np.sum((v1 - v2) ** 2))
```

```
def find_k_nearest_neighbors(train_docs, train_labels, test_doc, k, vocab_list):
```

"""

نزدیک‌ترین همسایه‌ها برای یک سند آزمون k پیدا کردن
"""

برای سند آزمون tf محاسبه

```
test_vector = np.zeros(len(vocab_list))
```

```
for j, term in enumerate(vocab_list):
    test_vector[j] = test_doc.count(term)
```

محاسبه فاصله از هر سند آزمون به تمام اسناد آموزشی

```
distances = []
```

```
for i, (doc, label) in enumerate(zip(train_docs, train_labels)):
```

تبدیل سند آموزشی به بردار tf

```
train_vector = np.zeros(len(vocab_list))
```

```
for j, term in enumerate(vocab_list):
    train_vector[j] = doc.count(term)
```

محاسبه فاصله اقلیدسی

```
distance = euclidean_distance(test_vector, train_vector)
```

```
distances.append((distance, label))
```

نزدیک‌ترین k مرتب‌سازی بر اساس فاصله و انتخاب

```
distances.sort(key=lambda x: x[0])
```

```
k_nearest = distances[:k]
```

نزدیک‌ترین همسایه‌ها k شمارش آرا برای هر کلاس در

```
class_votes = Counter([label for _, label in k_nearest])
```

انتخاب کلاس با بیشترین آرا

```
predicted_class = class_votes.most_common(1)[0][0]
```

```
return predicted_class, k_nearest
```

```
def classify_knn(train_docs, train_labels, test_docs, k):
```

"""

kNN طبقه‌بندی اسناد با استفاده از

"""

ایجاد واژگان مشترک

```
vocab = set()
```

```
for doc in train_docs + test_docs:
```

```
    vocab.update(doc)
```

```
vocab_list = list(vocab)
```

```
predictions = []
```

```
for test_doc in test_docs:
```

```
    predicted_class, _ = find_k_nearest_neighbors(train_docs, train_labels, test_doc, k, vocab_list)
```

```
    predictions.append(predicted_class)
```

```
return predictions
```

```
def evaluate_knn(train_docs, train_labels, test_docs, test_labels, k):
```

"""

kNN ارزیابی عملکرد

"""

```
predictions = classify_knn(train_docs, train_labels, test_docs, k)
```

```
correct = sum(1 for predicted, true_label in zip(predictions, test_labels) if predicted == true_label)
```

```
accuracy = correct / len(test_docs)
```

```
return accuracy
```

```
def plot_knn_decision_boundary(train_docs, train_labels, k=3):
```

"""

(Voronoi tessellation نسخه ساده از) kNN نمایش مرزهای تصمیم

"""

```
plt.figure(figsize=(10, 8))
```

ایجاد واژگان مشترک

```
vocab = set()
```

```
for doc in train_docs:
```

```
    vocab.update(doc)
```

```
vocab_list = list(vocab)
```

تبدیل اسناد به بردارهای tf

```
doc_vectors = []
```

```
for doc in train_docs:
```

```
    doc_vector = np.zeros(len(vocab_list))
```

```
    for j, term in enumerate(vocab_list):
```

```
        doc_vector[j] = doc.count(term)
```

```
    doc_vectors.append(doc_vector)
```

برای نمایش در فضای دو بعدی PCA کاهش ابعاد با

```
pca = PCA(n_components=2)
```

```
doc_vectors_2d = pca.fit_transform(doc_vectors)
```

نمایش اسناد آموزشی

```
colors = {'China': 'red', 'UK': 'blue', 'Kenya': 'green'}
```

```
for i, (vector_2d, label) in enumerate(zip(doc_vectors_2d, train_labels)):
```

```
    plt.scatter(vector_2d[0], vector_2d[1],
```

```
                c=colors[label], alpha=0.7, label=label)
```

```
unique_labels = list(set(train_labels))
label_to_numeric = {label: i for i, label in enumerate(unique_labels)}

# نمایش مرزهای تصمیم (فقط بین دو کلاس اول)
# ایجاد یک شبکه برای نمایش مرزهای تصمیم
x_min, x_max = doc_vectors_2d[:, 0].min() - 1, doc_vectors_2d[:, 0].max() + 1
y_min, y_max = doc_vectors_2d[:, 1].min() - 1, doc_vectors_2d[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                     np.arange(y_min, y_max, 0.1))

# طبقه‌بندی هر نقطه در شبکه
Z = []
for point in np.c_[xx.ravel(), yy.ravel()]:
    # بازگشت به فضای اصلی با استفاده از معکوس PCA
    point_original = pca.inverse_transform(point.reshape(1, -1))

    # پیدا کردن نزدیک‌ترین همسایه
    min_distance = float('inf')
    nearest_label = None
    for i, (vector, label) in enumerate(zip(doc_vectors, train_labels)):
        distance = euclidean_distance(point_original, vector)
        if distance < min_distance:
            min_distance = distance
            nearest_label = label

    Z.append(label_to_numeric[nearest_label])

Z = np.array(Z).reshape(xx.shape)

cmap = plt.cm.colors.ListedColormap([colors[label] for label in unique_labels])

# نمایش مرزهای تصمیم
plt.contourf(xx, yy, Z, alpha=0.2, cmap=cmap)

plt.xlabel('first principal component')
plt.ylabel('second principal component')
plt.title('Decision boundary of kNN (k=3)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.show()

# مثال کاربردی: طبقه‌بندی اخبار مربوط به کشورها
documents = [
    ["Chinese", "Beijing", "Mao"],
    ["Chinese", "Shanghai", "Mao"],
    ["Chinese", "Taiwan", "Mao"],
    ["Chinese", "Tokyo", "Mao"],
    ["British", "London", "Queen"],
    ["British", "London", "Queen"],
    ["Kenyan", "Nairobi", "tourism"],
    ["Kenyan", "Nairobi", "safari"]
]
labels = ["China", "China", "China", "China", "UK", "UK", "Kenya", "Kenya"]

# k=1 طبقه‌بندی با
predicted_1nn = classify_knn(documents, labels, documents, 1)
print(f"پیش‌بینی kNN (k=1): {predicted_1nn}")

# k=3 طبقه‌بندی با
predicted_3nn = classify_knn(documents, labels, documents, 3)
print(f"پیش‌بینی kNN (k=3): {predicted_3nn}")

# ارزیابی عملکرد
accuracy_1nn = evaluate_knn(documents, labels, documents, labels, 1)
accuracy_3nn = evaluate_knn(documents, labels, documents, labels, 3)
print(f"\nدقت kNN (k=1): {accuracy_1nn:.2f}")
print(f"دقت kNN (k=3): {accuracy_3nn:.2f}")

# نمایش مرزهای تصمیم
plot_knn_decision_boundary(documents, labels, k=3)

# kNN نمایش پیچیدگی‌های
print("\nپیچیدگی‌های kNN:")
print("1. حساسیت بالا در مجموعه‌های بزرگ: برای مجموعه‌های بزرگ، محاسبه فاصله برای هر سند آزمون ممکن است زمان‌بر باشد.")
print("2. به حافظه نیاز دارد تا تمام اسناد آموزشی را ذخیره کند: kNN حساسیت ذخیره: حساسیت ذخیره.")
print("3. عدم تعادل وزن‌دهی ویژگی‌ها: ویژگی‌هایی با مقادیر بزرگ ممکن است بر ویژگی‌های با مقادیر کوچک تأثیر بیشتری داشته باشند.")

# نمایش پیچیدگی زمانی
print("\nپیچیدگی زمانی kNN:")
print("به صورت زیر محاسبه می‌شود: kNN پیچیدگی زمانی")
print("O(|D|L<sub>ave</sub> + |C||V|)")
print("تعداد ویژگی‌ها است |V| تعداد کلاس‌ها و |C|، میانگین تعداد توکن در هر سند <sub>ave</sub>L، تعداد اسناد |D| که در آن")
print("خطی در اندازه مجموعه آموزشی است kNN این نشان می‌دهد که پیچیدگی زمانی")
print("بدر است زیرا نیاز به محاسبه و ذخیره تمام اسناد آموزشی دارد Rocchio معمولاً از kNN در مقایسه، پیچیدگی زمانی")

# مثال واقعی: سیستم اسپاتیفای
print("\nمثال واقعی: سیستم اسپاتیفای")
print("برای پیدا کردن موسیقی مشابه بر اساس ویژگی‌های صوتی بسیار مفید است kNN، در سیستم اسپاتیفای")
print("این سیستم می‌تواند دو آهنگ با ریتم‌های متفاوت را به عنوان \مشابه" شناسایی کند، حتی اگر متون آن‌ها کاملاً متفاوت باشند")
print("این کار به ویژگی‌های صوتی که مستقل از زبان و فرهنگ موسیقی هستند تکیه می‌دهد")

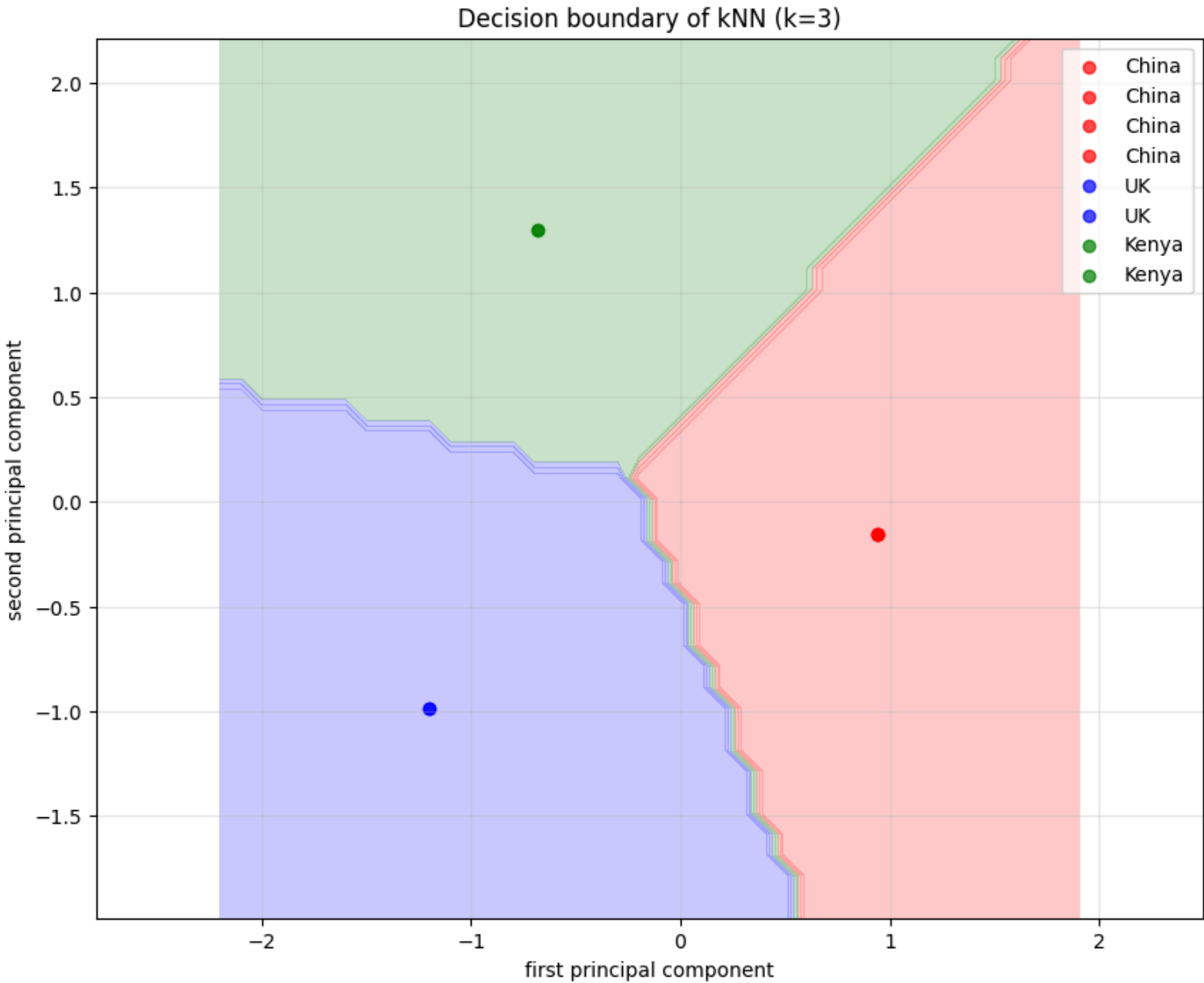
# نمایش پیچیدگی زمانی
print("\nپیچیدگی زمانی: چگونه کارایی را بهبود آوریم؟")
print("1. برای جستجوی سریع‌تر KD-tree بهینه‌سازی ساختار داده: استفاده از ساختارهای داده کارآمد مانند")
print("2. به تعداد معقولی برای کاهش محاسبات k استفاده از تقریب‌دهی تقریبی: محدود کردن")
```

```
print("3. استفاده از فاصله‌های جایگزین مانند فاصله کسینوسی که به وزن‌دهی ویژگی‌ها توجه بیشتری به موارد مهم می‌دهد.")
print("4. که به ویژگی‌های با وزن کمتر وزن بیشتری می‌دهد Local Sensitive Hashing (LSH) استفاده از روش‌های احتمالی مانند")
```

kNN (k=1): ['China', 'China', 'China', 'China', 'UK', 'UK', 'Kenya', 'Kenya']
پیش‌بینی
kNN (k=3): ['China', 'China', 'China', 'China', 'UK', 'UK', 'Kenya', 'Kenya']
پیش‌بینی

kNN (k=1): 1.00
دقت

kNN (k=3): 1.00
دقت



kNN: پیچیدگی‌های

- حساسیت بالا در مجموعه‌های بزرگ: برای مجموعه‌های بزرگ، محاسبه فاصله برای هر سند آزمون ممکن است زمان‌بر باشد.
- به حافظه نیاز دارد تا تمام اسناد آموزشی را ذخیره کند kNN: حساسیت ذخیره.
- عدم تعادل وزن‌دهی ویژگی‌ها: ویژگی‌هایی با مقادیر بزرگ ممکن است بر ویژگی‌های با مقادیر کوچک تأثیر بیشتری داشته باشند.

kNN: پیچیدگی زمانی

:به صورت زیر محاسبه می‌شود kNN پیچیدگی زمانی

$\Theta(|D|L_{\text{ave}} + |C||V|)$

.تعداد ویژگی‌ها است $|V|$ تعداد کلاس‌ها و $|C|$ ، میانگین تعداد توکن در هر سند L_{ave} ، تعداد اسناد $|D|$ که در آن

.خطی در اندازه مجموعه آموزشی است kNN این نشان می‌دهد که پیچیدگی زمانی

.بدتر است زیرا نیاز به محاسبه و ذخیره تمام اسناد آموزشی دارد Rocchio معمولاً از kNN در مقایسه، پیچیدگی زمانی

مثال واقعی: سیستم اسپاتیفای

.برای پیدا کردن موسیقی مشابه بر اساس ویژگی‌های صوتی بسیار مفید است kNN، در سیستم اسپاتیفای

.این سیستم می‌تواند دو آهنگ با ریتم‌های متفاوت را به عنوان "مشابه" شناسایی کند، حتی اگر متون آن‌ها کاملاً متفاوت باشند

.این کار به ویژگی‌های صوتی که مستقل از زبان و فرهنگ موسیقی هستند تکیه می‌دهد

پیچیدگی زمانی: چگونه کارایی را بهبود آوریم؟

- .برای جستجوی سریع‌تر KD-tree بهینه‌سازی ساختار داده: استفاده از ساختارهای داده کارآمد مانند
- .به تعداد معقولی برای کاهش محاسبات k استفاده از تقریبندی تقریبی: محدود کردن
- .استفاده از فاصله‌های جایگزین مانند فاصله کسینوسی که به وزن‌دهی ویژگی‌ها توجه بیشتری به موارد مهم می‌دهد.
- .که به ویژگی‌های با وزن کمتر وزن بیشتری می‌دهد Local Sensitive Hashing (LSH) استفاده از روش‌های احتمالی مانند

14.3.1 پیچیدگی زمانی و بهینگی kNN: چالش‌های عملی در دنیای واقعی 🕒

🔍 Netflix و چالش توصیه فیلم: چگونه فیلم‌های مشابه را به سرعت پیدا کنیم؟

تصور کنید در تیم توصیه فیلم Netflix کار می‌کنید و باید سیستمی طراحی کنید که به کاربران فیلم‌های مشابه بر اساس علاقه‌مندی‌هایشان پیشنهاد دهد. با میلیون‌ها کاربر و هزاران فیلم، چگونه می‌توانید به سرعت k نزدیک‌ترین همسایه‌ها را برای هر کاربر پیدا کنید؟ این دقیقاً همان چالشی است که kNN در مقیاس بزرگ با آن روبرو است.

در حالی که kNN یک روش بسیار قدرتمند است، اما پیچیدگی زمانی آن می‌تواند یک چالش جدی برای سیستم‌های بزرگ باشد. بیا ببینیم چگونه می‌توانیم این چالش را مدیریت کنیم.

📌 پیچیدگی زمانی kNN: آموزش سریع، تست زمان‌بر

جدول 14.3 پیچیدگی زمانی kNN را نشان می‌دهد. ویژگی‌های kNN کاملاً با اکثر الگوریتم‌های طبقه‌بندی دیگر متفاوت است:

kNN with preprocessing of training set	
training	$\Theta(D L_{ave})$
testing	$\Theta(L_a + D M_{ave}M_a) = \Theta(D M_{ave}M_a)$
kNN without preprocessing of training set	
training	$\Theta(1)$
testing	$\Theta(L_a + D L_{ave}M_a) = \Theta(D L_{ave}M_a)$

► Table 14.3 Training and test times for kNN classification. M_{ave} is the average size of the vocabulary of documents in the collection.

- **زمان آموزش:** آموزش یک طبقه‌بند kNN صرفاً شامل تعیین k و پیش‌پردازش اسناد است. در واقع، اگر مقدار k را از قبل انتخاب کنیم و پیش‌پردازش انجام ندهیم، kNN اصلاً نیازی به آموزش ندارد!
- **زمان تست:** زمان تست برای kNN برابر با $\Theta(|D|M_{ave}M_a)$ است. این زمان خطی با اندازه مجموعه آموزشی است، زیرا باید فاصله هر سند آموزشی از سند تست را محاسبه کنیم.
- **مزیت برای کلاس‌های زیاد:** زمان تست مستقل از تعداد کلاس‌های L است. بنابراین kNN برای مسائلی با L بزرگ یک مزیت بالقوه دارد.

مثال واقعی: در سیستم توصیه فیلم Amazon Prime، زمان آموزش kNN تقریباً صفر است، زیرا سیستم فقط نیاز به ذخیره تمام بردارهای ویژگی فیلم دارد. اما وقتی کاربر جدیدی وارد سیستم می‌شود، برای پیشنهاد 5 فیلم برتر، باید فاصله بین بردار ویژگی کاربر و تمام فیلم‌ها را محاسبه کند که با میلیون‌ها فیلم در مجموعه، این کار می‌تواند زمان‌بر باشد.

📁 یادگیری مبتنی بر حافظه: نقاط قوت و ضعف

در طبقه‌بندی kNN، برخلاف طبقه‌بندی Rocchio (مراکز) یا بیز ساده (اولویت‌ها و احتمالات شرطی)، هیچ تخمین پارامتری انجام نمی‌دهیم. kNN صرفاً تمام مثال‌های مجموعه آموزشی را به خاطر می‌سپارد و سپس سند تست را با آن‌ها مقایسه می‌کند. به همین دلیل، kNN همچنین یادگیری مبتنی بر حافظه یا یادگیری مبتنی بر نمونه نامیده می‌شود.

مثال واقعی: در سیستم تشخیص چهره فیس‌بوک، kNN می‌تواند چهره یک فرد را با مقایسه با تمام چهره‌های موجود در پایگاه داده شناسایی کند. این سیستم نیازی به استخراج ویژگی‌های کلی از چهره‌ها ندارد، بلکه مستقیماً تصاویر را با هم مقایسه می‌کند. این کار دقت بالایی دارد، اما با افزایش تعداد کاربران، نیاز به حافظه و قدرت پردازش بیشتر می‌شود.

🚀 بهینه‌سازی kNN: چگونه می‌توانیم کارایی را بهبود دهیم؟

آیا می‌توان تست kNN را کارآمدتر از $\Theta(|D| \text{MaveMa})$ یا نادیده گرفتن طول اسناد، کارآمدتر از $\Theta(|D|)$ کرد؟

- **الگوریتم‌های سریع برای ابعاد کوچک:** الگوریتم‌های سریع kNN برای ابعاد کوچک M وجود دارد (تمرین 14.12).
- **تقریب‌ها برای ابعاد بزرگ:** تقریب‌هایی برای M بزرگ وجود دارد که کران‌های خطا برای بهره‌وری مشخصی ارائه می‌دهند (بخش 14.7).
- **استفاده از نمایه معکوس:** شباهت بین پیدا کردن نزدیک‌ترین همسایگان یک سند تست و بازیابی ad hoc، جایی که ما به دنبال اسناد با بیشترین شباهت به کوئری هستیم، قابل توجه است.

مثال واقعی: در سیستم جستجوی گوگل، برای پیدا کردن صفحات وب مشابه با یک کوئری، از نمایه معکوس استفاده می‌شود تا جستجو را به اسنادی محدود کند که حداقل یک کلمه مشترک با کوئری دارند. این رویکرد می‌تواند زمان جستجو را به طور قابل توجهی کاهش دهد، به خصوص برای کوئری‌های کوتاه که با استفاده از لیست متوقف کلمات بزرگ، همپوشانی کمی با اسناد دارند.

🇮🇷 بهینگی kNN: چقدر دقیق است؟

همانطور که در فصل بعد خواهیم دید، اثربخشی kNN نزدیک به دقیق‌ترین روش‌های یادگیری در طبقه‌بندی متن است (جدول 15.2، صفحه 334). یک معیار برای کیفیت یک روش یادگیری، نرخ خطای بیز آن است، یعنی میانگین نرخ خطای طبقه‌بندی یادگرفته شده توسط آن برای یک مسئله خاص.

kNN برای مسائلی با نرخ خطای بیز غیرصفر بهینه نیست - یعنی مسائلی که حتی بهترین طبقه‌بند ممکن نرخ خطای طبقه‌بندی غیرصفر دارد. خطای 1NN به صورت مجانبی (با افزایش مجموعه آموزشی) توسط دو برابر نرخ خطای بیز محدود می‌شود. یعنی اگر طبقه‌بند بهینه نرخ خطای x داشته باشد، 1NN نرخ خطای مجانبی کمتر از $2x$ خواهد داشت.

مثال واقعی: در سیستم تشخیص اسپم جیمیل، حتی بهترین طبقه‌بند ممکن است به دلیل تکنیک‌های پیچیده اسپمرها، نرخ خطای 1% داشته باشد. طبقه‌بند 1NN در این حالت ممکن است نرخ خطای تا 2% داشته باشد، که هنوز برای کاربران قابل قبول است. اما برای مسائل با نرخ خطای بیز صفر (مانند تشخیص اعداد دستنویس در شرایط ایده‌آل)، خطای 1NN با افزایش اندازه مجموعه آموزشی به صفر نزدیک می‌شود.

14.4 طبقه‌بندی خطی در مقابل غیرخطی: مرزهای ساده در برابر

🇮🇷 پیچیدگی‌های دنیای واقعی

🎬 چالش طبقه‌بندی فیلم در نتفلیکس: یک خط ساده کافی است؟

تصور کنید در تیم طبقه‌بندی محتوای نتفلیکس (Netflix) کار می‌کنید. وظیفه شما این است که فیلم‌ها را به دو دسته "اکشن" و "کمدی" تقسیم کنید. شاید بتوانید با ویژگی‌هایی مانند "تعداد انفجارها در دقیقه" یا "تعداد

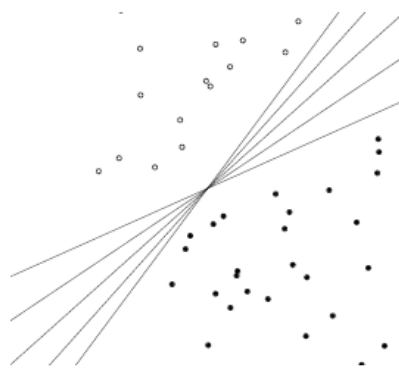
جوک‌ها در دقیقه" یک خط مرزی ساده بین این دو ژانر رسم کنید. این یک **طبقه‌بند خطی** است.

حالا تصور کنید وظیفه شما پیشنهاد فیلم به کاربران است. مرز بین "فیلم‌هایی که کاربر دوست دارد" و "فیلم‌هایی که دوست ندارد" بسیار پیچیده و شخصی است. آیا یک خط ساده می‌تواند این تمایز را ایجاد کند؟ به احتمال زیاد خیر. این یک **مسئله غیرخطی** است که به الگوریتم‌های پیچیده‌تری مانند kNN نیاز دارد.

در این بخش، با تفاوت طبقه‌بندهای خطی و غیرخطی آشنا می‌شویم. **طبقه‌بند خطی** طبقه‌بندی را با مقایسه یک ترکیب خطی از ویژگی‌ها با یک آستانه (threshold) انجام می‌دهد. در فضای دو بعدی، این طبقه‌بند یک خط است و در فضاهای با ابعاد بالاتر، یک هایپرپلن.

شکل 14.8: بی‌نهایت خط برای جدا کردن دو کلاس

اگر دو کلاس کاملاً از هم جدا باشند (linearly separable)، بی‌نهایت خط (یا هایپرپلن) می‌تواند آن‌ها را از هم جدا کند. چالش اصلی، انتخاب بهترین خط است که روی داده‌های جدید بهترین عملکرد را داشته باشد.



► Figure 14.8 There are an infinite number of hyperplanes that separate two linearly separable classes.

الگوریتم یک طبقه‌بند خطی بسیار ساده است: یک امتیاز (score) بر اساس مجموع وزن‌دار ویژگی‌ها محاسبه شده و با آستانه مقایسه می‌شود.

شکل 14.9: الگوریتم طبقه‌بندی خطی

این الگوریتم نشان می‌دهد که چگونه یک بردار ویژگی ($\vec{x}$) با استفاده از بردار وزن‌ها ($\vec{w}$) و آستانه (b) طبقه‌بندی می‌شود.

```
APPLYLINEARCLASSIFIER( $\vec{w}, b, \vec{x}$ )
1   $score \leftarrow \sum_{i=1}^M w_i x_i$ 
2  if  $score > b$ 
3    then return 1
4    else return 0
```

► Figure 14.9 Linear classification algorithm.

اگر مجموع وزن دار ویژگی‌ها (score) از آستانه **b** بیشتر باشد، سند به کلاس 1 و در غیر این صورت به کلاس 0 تعلق می‌گیرد. چالش واقعی در آموزش این طبقه‌بند است، یعنی پیدا کردن مقادیر بهینه برای w و b .

شاید تعجب‌آور باشد، اما الگوریتم‌هایی که تاکنون بررسی کردیم، یعنی **Rocchio** و **Naive Bayes**، هر دو نمونه‌هایی از طبقه‌بندهای خطی هستند.

- **Rocchio**: مرز تصمیم در Rocchio، مجموعه‌ای از نقاط است که فاصله آن‌ها از دو مرکز (centroid) کلاس‌ها برابر است. می‌توان ثابت کرد که این مرز یک هایپرپلن است.
- **Naive Bayes**: قانون تصمیم Naive Bayes بر اساس بیشترین مقدار $P(c|d)$ است. اگر از لگاریتم نسبت احتمالات (log odds) استفاده کنیم، معادله به صورت یک ترکیب خطی درمی‌آید:

$$\log P(c|d) / P(c^-|d) = \log P(c) / P(c^-) + \sum \log P(t_k|c) / P(t_k|c^-)$$

این معادله دقیقاً همان ساختار یک طبقه‌بند خطی ($wT \sim x > b$) را دارد.

جدول 14.4: یک طبقه‌بند خطی برای دسته‌بندی اخبار اقتصادی

این جدول یک طبقه‌بند خطی واقعی برای دسته "interest" (نرخ بهره) در مجموعه داده Reuters-21578 را نشان می‌دهد. هر کلمه یک وزن (w_i) دارد. کلماتی مانند "rate" و "discount" وزن مثبت دارند (نشانه‌ای از دسته interest)، در حالی که کلماتی مانند "dlrs" و "world" وزن منفی دارند (احتمالاً نشانه‌ای از دسته‌های دیگر مانند "currency").

t_i	w_i	d_{1i}	d_{2i}	t_i	w_i	d_{1i}	d_{2i}
prime	0.70	0	1	dlrs	-0.71	1	1
rate	0.67	1	0	world	-0.35	1	0
interest	0.63	0	0	sees	-0.33	0	0
rates	0.60	0	0	year	-0.25	0	0
discount	0.46	1	0	group	-0.24	0	0
bundesbank	0.43	0	0	dlr	-0.24	0	0

► Table 14.4 A linear classifier. The dimensions t_i and parameters w_i of a linear classifier for the class *interest* (as in interest rate) in Reuters-21578. The threshold is $b = 0$. Terms like dlr and world have negative weights because they are indicators for the competing class *currency*.

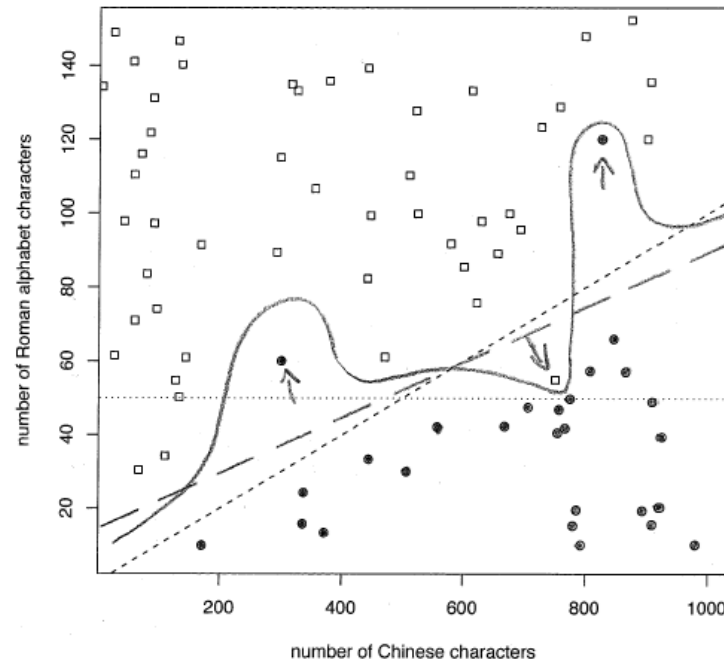
برای طبقه‌بندی یک سند جدید، کفایت وزن کلمات موجود در سند را با هم جمع کنیم. اگر حاصل از آستانه ($b=0$) بیشتر باشد، سند به دسته "interest" تعلق دارد.

چالش‌های طبقه‌بندهای خطی: نویز و جدایی خطی

طبقه‌بندهای خطی با چالش‌های خاصی روبرو هستند:

- **اسناد نویزی (Noise Documents)**: اسنادی وجود دارند که در منطقه‌ای از فضا قرار دارند که اکثر اسناد متعلق به کلاس دیگری هستند. این اسناد می‌توانند الگوریتم آموزش را گمراه کنند. در شکل 14.10، سه سند با علامت پیکان، اسناد نویزی هستند.
- **جدایی خطی (Linear Separability)**: اگر یک هایپرپلن بتواند دو کلاس را به طور کامل از هم جدا کند، می‌گوییم کلاس‌ها "خطی جدایی‌پذیر" هستند. در این صورت، بی‌نهایت هایپرپلن جداکننده وجود

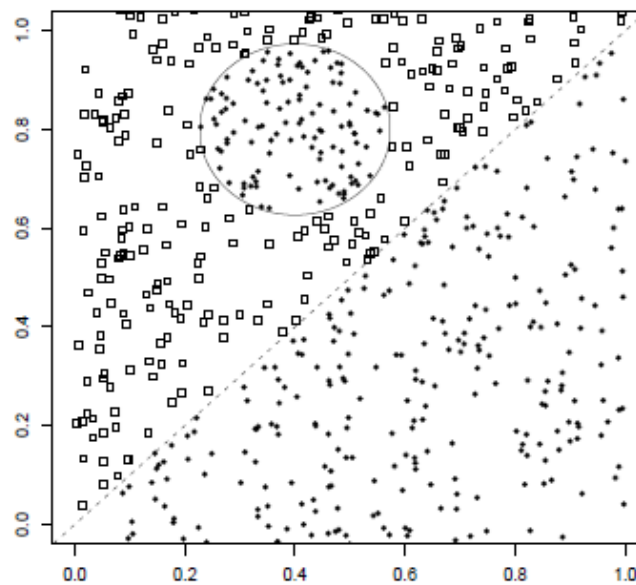
دارد (مانند شکل 14.8). چالش این است که بهترین هایپرپلن را انتخاب کنیم که روی داده‌های جدید بهترین عملکرد را داشته باشد.



► Figure 14.10 A linear problem with noise. In this hypothetical web page classification scenario, Chinese-only web pages are solid circles and mixed Chinese-English web pages are squares. The two classes are separated by a linear class boundary (dashed line, short dashes), except for three noise documents (marked with arrows).

🌀 طبقه‌بندهای غیرخطی: زمانی که یک خط کافی نیست

گاهی اوقات، مرز بین دو کلاس به سادگی یک خط یا هایپرپلن نیست. شکل 14.11 یک مسئله غیرخطی را نشان می‌دهد. یک طبقه‌بند خطی نمی‌تواند "clave" دایره‌ای شکل در گوشه بالا سمت چپ را به درستی طبقه‌بندی کند.



► Figure 14.11 A nonlinear problem.

در این موارد، طبقه‌بندهای غیرخطی مانند kNN عملکرد بهتری دارند. مرز تصمیم kNN (خطوط دوتایی در شکل 14.6) از قطعات خطی محلی تشکیل شده است، اما به طور کلی شکلی پیچیده دارد که معادل یک خط یا هایپرپلن واحد نیست.

مثال واقعی: در سیستم تشخیص چهره فیس‌بوک، مرز بین "چهره شما" و "چهره افراد دیگر" بسیار پیچیده

است. یک طبقه‌بند خطی ساده نمی‌تواند این تمایز را با دقت بالا ایجاد کند. در مقابل، kNN می‌تواند با مقایسه

یک چهره جدید با نزدیک‌ترین چهره‌های موجود در پایگاه داده، این کار را با دقت بالایی انجام دهد.

در نهایت، انتخاب بین طبقه‌بند خطی و غیرخطی به ماهیت مسئله بستگی دارد. اگر مسئله خطی است، یک طبقه‌بند

خطی ساده و کارآمد بهترین انتخاب است. اما اگر مرزهای کلاس‌ها پیچیده و غیرخطی هستند، طبقه‌بندهای

غیرخطی مانند kNN دقت بالاتری خواهند داشت.

In [11]: پیاده‌سازی و مقایسه طبقه‌بندهای خطی و غیرخطی #

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# تولید داده‌های خطی جداشد با نویز
def generate_linear_data(n_samples=200, noise_level=0.1):
    """
    تولید داده‌های خطی جداشد با نویز برای شبیه‌سازی چالش‌های دنیای واقعی
    """
    np.random.seed(42)

    # ایجاد دو کلاس خطی جداشد
    X_class1 = np.random.randn(n_samples, 2) + np.array([2, 2])
    X_class2 = np.random.randn(n_samples, 2) + np.array([-2, -2])

    # ایجاد برچسب‌ها
    y_class1 = np.ones(n_samples)
    y_class2 = np.zeros(n_samples)

    # ترکیب داده‌ها
    X = np.vstack([X_class1, X_class2])
    y = np.hstack([y_class1, y_class2])

    # افزودن نویز به درصدی از داده‌ها (اسناد نویزی)
    n_noise = int(noise_level * n_samples)
    noise_indices = np.random.choice(len(X), n_noise, replace=False)

    # معکوس کردن برچسب برخی از نقاط نویز
    y[noise_indices] = 1 - y[noise_indices]

    return X, y

# تولید داده‌های غیرخطی
def generate_nonlinear_data(n_samples=200):
    """
    تولید داده‌های واقعاً غیرخطی برای نمایش محدودیت‌های طبقه‌بندهای خطی
    """
    np.random.seed(42)

    # ایجاد داده‌های شکل ماه (غیرخطی)
    n = n_samples // 2
    X1 = np.zeros((n, 2))
    X2 = np.zeros((n, 2))

    # کلاس اول: نیمه بالایی ماه
    for i in range(n):
        angle = np.pi * i / n
        X1[i, 0] = np.cos(angle)
        X1[i, 1] = np.sin(angle)

    # کلاس دوم: نیمه پایینی ماه (بدون شیب رو به پایین)
    for i in range(n):
        angle = np.pi + np.pi * i / n
        X2[i, 0] = np.cos(angle)
        X2[i, 1] = np.sin(angle) # حذف شیب رو به پایین

    # ترکیب داده‌ها
    X = np.vstack([X1, X2])
    y = np.hstack([np.zeros(n), np.ones(n)])

    return X, y

# Rocchio پیاده‌سازی طبقه‌بند خطی
class RocchioClassifier:
    """
    که یک طبقه‌بند خطی است Rocchio پیاده‌سازی ساده طبقه‌بند
    """
    def __init__(self):
        self.centroids = {}

    def fit(self, X, y):
        """
        برای هر کلاس oids محاسبه مرکز
        """
        pass
```

```

"""
# دریافت کلاس‌های یکتا
classes = np.unique(y)

# محاسبه مرکز برای هر کلاس
for c in classes:
    # انتخاب تمام نقاط داده برای کلاس c
    X_c = X[y == c]
    # محاسبه میانگین (مرکز)
    self.centroids[c] = np.mean(X_c, axis=0)

def predict(self, X):
    """
    پیش‌بینی کلاس برای هر نقطه داده بر اساس نزدیک‌ترین مرکز
    """
    y_pred = []

    # برای هر نقطه داده
    for x in X:
        # مقداردهی اولیه با فاصله بزرگ
        min_dist = float('inf')
        best_class = None

        # برای هر مرکز کلاس
        for c, centroid in self.centroids.items():
            # محاسبه فاصله تا مرکز
            dist = np.linalg.norm(x - centroid)

            # اگر این مرکز نزدیک‌تر است، به‌روزرسانی کن
            if dist < min_dist:
                min_dist = dist
                best_class = c

        # افزودن بهترین کلاس به پیش‌بینی‌ها
        y_pred.append(best_class)

    return np.array(y_pred)

# Naive Bayes پیاده‌سازی طبقه‌بند خطی
class NaiveBayesClassifier:
    """
    که یک طبقه‌بند خطی است Naive Bayes پیاده‌سازی ساده طبقه‌بند
    """
    def __init__(self):
        self.class_priors = {}
        self.class_feature_means = {}
        self.class_feature_vars = {}

    def fit(self, X, y):
        """
        محاسبه احتمالات اولیه کلاس و توزیع ویژگی‌ها
        """
        # دریافت کلاس‌های یکتا
        classes = np.unique(y)

        # محاسبه احتمال اولیه برای هر کلاس
        for c in classes:
            # مجموع_تعداد / تعداد(c) = احتمال اولیه
            self.class_priors[c] = np.mean(y == c)

            # محاسبه میانگین و واریانس برای هر ویژگی
            X_c = X[y == c]
            self.class_feature_means[c] = np.mean(X_c, axis=0)
            self.class_feature_vars[c] = np.var(X_c, axis=0) + 1e-9 # افزودن مقدار کوچک برای جلوگیری از تقسیم بر صفر

    def predict(self, X):
        """
        Naive Bayes پیش‌بینی کلاس برای هر نقطه داده با استفاده از
        """
        y_pred = []

        # برای هر نقطه داده
        for x in X:
            # مقداردهی اولیه با مقدار بزرگ منفی (فضای لگاریتمی)
            max_log_prob = -float('inf')
            best_class = None

            # برای هر کلاس
            for c in self.class_priors:
                # شروع با لگاریتم احتمال اولیه
                log_prob = np.log(self.class_priors[c])

                # افزودن لگاریتم احتمال برای هر ویژگی
                for i in range(len(x)):
                    # محاسبه احتمال مقدار ویژگی با توجه به کلاس
                    # با استفاده از توزیع گوسی
                    mean = self.class_feature_means[c][i]
                    var = self.class_feature_vars[c][i]

                    # لگاریتم احتمال مقدار ویژگی
                    log_prob += -0.5 * np.log(2 * np.pi * var) - 0.5 * ((x[i] - mean) ** 2) / var

                # اگر این کلاس احتمال بالاتری دارد، به‌روزرسانی کن
                if log_prob > max_log_prob:
                    max_log_prob = log_prob
                    best_class = c

```

```

        # افزودن بهترین کلاس به پیش‌بینی‌ها
        y_pred.append(best_class)

    return np.array(y_pred)

# پیاده‌سازی طبقه‌بند غیرخطی
class KNNClassifier:
    """
    نزدیک‌ترین همسایه که یک طبقه‌بند غیرخطی است-k پیاده‌سازی ساده طبقه‌بند
    """

    def __init__(self, k=5):
        self.k = k
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        """
        (یک یادگیرنده تنبل است kNN) ذخیره داده‌های آموزشی
        """
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        """
        kNN پیش‌بینی کلاس برای هر نقطه داده با استفاده از
        """
        y_pred = []

        # برای هر نقطه داده
        for x in X:
            # محاسبه فاصله تا تمام نقاط آموزشی
            distances = np.array([np.linalg.norm(x - x_train) for x_train in self.X_train])

            # نزدیک‌ترین همسایه‌ها k دریافت شاخص‌های
            k_nearest_indices = np.argsort(distances)[:self.k]

            # نزدیک‌ترین همسایه‌ها k دریافت برچسب‌های
            k_nearest_labels = self.y_train[k_nearest_indices]

            # پیش‌بینی با رأی اکثریت
            counts = np.bincount(k_nearest_labels.astype(int))
            y_pred.append(np.argmax(counts))

        return np.array(y_pred)

# تابع برای رسم مرز تصمیم
def plot_decision_boundary(classifier, X, y, title, ax=None):
    """
    رسم مرز تصمیم یک طبقه‌بند
    """

    if ax is None:
        ax = plt.gca()

    # ایجاد یک شبکه مش
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                          np.arange(y_min, y_max, 0.1))

    # پیش‌بینی برای هر نقطه در شبکه مش
    Z = classifier.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # نمایش مرز تصمیم و مناطق
    ax.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Paired)
    ax.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k')

    ax.set_title(title)
    ax.set_xlabel('Feature 1')
    ax.set_ylabel('Feature 2')

# تابع برای مقایسه طبقه‌بندها روی داده‌های خطی
def compare_linear_classifiers():
    """
    مقایسه طبقه‌بندهای خطی و غیرخطی روی داده‌های خطی جادش با نویز
    """

    # تولید داده‌های خطی با نویز
    X, y = generate_linear_data(200, noise_level=0.1)

    # تقسیم داده‌ها
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    # ایجاد طبقه‌بندها
    rocchio = RocchioClassifier()
    naive_bayes = NaiveBayesClassifier()
    knn = KNNClassifier(k=5)

    # آموزش طبقه‌بندها
    rocchio.fit(X_train, y_train)
    naive_bayes.fit(X_train, y_train)
    knn.fit(X_train, y_train)

    # ارزیابی طبقه‌بندها
    rocchio_acc = accuracy_score(y_test, rocchio.predict(X_test))
    naive_bayes_acc = accuracy_score(y_test, naive_bayes.predict(X_test))
    knn_acc = accuracy_score(y_test, knn.predict(X_test))

```

```
print(f"Rocchio Accuracy: {rocchio_acc:.3f}")
print(f"Naive Bayes Accuracy: {naive_bayes_acc:.3f}")
print(f"kNN Accuracy: {knn_acc:.3f}")

# نمایش مرزهای تصمیم
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

plot_decision_boundary(rocchio, X, y, 'Rocchio (Linear)', axes[0])
plot_decision_boundary(naive_bayes, X, y, 'Naive Bayes (Linear)', axes[1])
plot_decision_boundary(knn, X, y, 'kNN (Nonlinear)', axes[2])

plt.tight_layout()
plt.show()

# تابع برای مقایسه طبقه‌بندها روی داده‌های غیرخطی
def compare_nonlinear_classifiers():
    """
    مقایسه طبقه‌بندهای خطی و غیرخطی روی داده‌های غیرخطی جداشد
    """
    # تولید داده‌های غیرخطی
    X, y = generate_nonlinear_data(200)

    # تقسیم داده‌ها
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    # ایجاد طبقه‌بندها
    rocchio = RocchioClassifier()
    naive_bayes = NaiveBayesClassifier()
    knn = KNNClassifier(k=5)

    # آموزش طبقه‌بندها
    rocchio.fit(X_train, y_train)
    naive_bayes.fit(X_train, y_train)
    knn.fit(X_train, y_train)

    # ارزیابی طبقه‌بندها
    rocchio_acc = accuracy_score(y_test, rocchio.predict(X_test))
    naive_bayes_acc = accuracy_score(y_test, naive_bayes.predict(X_test))
    knn_acc = accuracy_score(y_test, knn.predict(X_test))

    print(f"Rocchio Accuracy: {rocchio_acc:.3f}")
    print(f"Naive Bayes Accuracy: {naive_bayes_acc:.3f}")
    print(f"kNN Accuracy: {knn_acc:.3f}")

    # نمایش مرزهای تصمیم
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    plot_decision_boundary(rocchio, X, y, 'Rocchio (Linear)', axes[0])
    plot_decision_boundary(naive_bayes, X, y, 'Naive Bayes (Linear)', axes[1])
    plot_decision_boundary(knn, X, y, 'kNN (Nonlinear)', axes[2])

    plt.tight_layout()
    plt.show()

# اجرای مقایسه روی داده‌های خطی
print("Comparison of Classifiers on Linear Data with Noise:")
print("=*50)
compare_linear_classifiers()

# اجرای مقایسه روی داده‌های غیرخطی
print("\nComparison of Classifiers on Non-linear Data:")
print("=*50)
compare_nonlinear_classifiers()
```

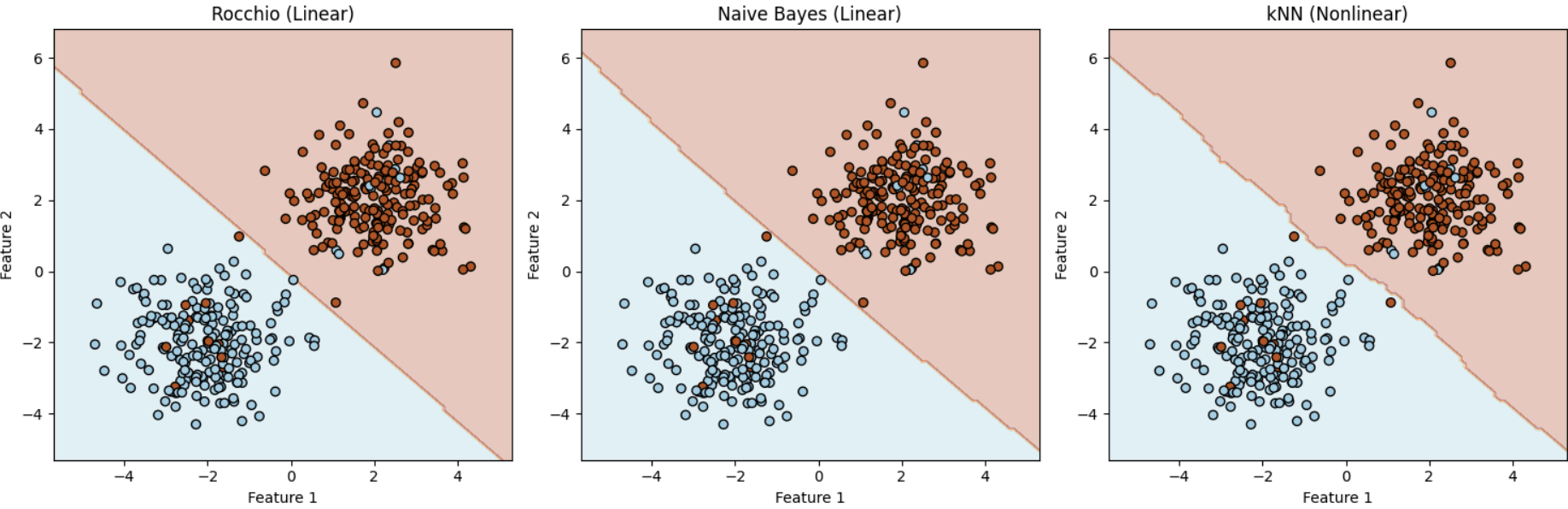
Comparison of Classifiers on Linear Data with Noise:

=====

Rocchio Accuracy: 0.933

Naive Bayes Accuracy: 0.933

kNN Accuracy: 0.925



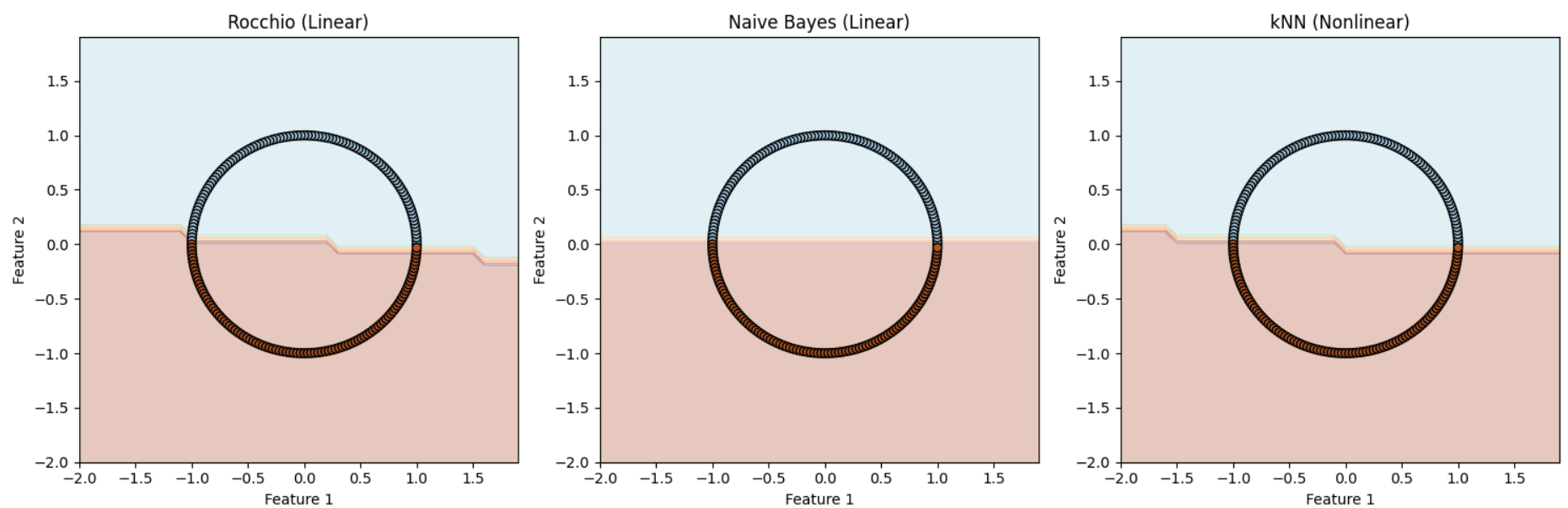
Comparison of Classifiers on Non-linear Data:

=====

Rocchio Accuracy: 0.983

Naive Bayes Accuracy: 1.000

kNN Accuracy: 1.000



14.5 طبقه‌بندی با بیش از دو کلاس: از اخبار چندبعدی تا توصیه‌های شخصی‌سازی شده 📺

🌐 چالش طبقه‌بندی خبر در گوگل نیوز: یک خبر می‌تواند چند برچسب داشته باشد؟

تصور کنید در تیم طبقه‌بندی محتوای گوگل نیوز کار می‌کنید و باید سیستمی طراحی کنید که اخبار را به دسته‌های مختلفی مانند "سیاست"، "اقتصاد"، "فناوری" و "ورزشی" تقسیم کند. آیا یک خبر می‌تواند همزمان به چند دسته تعلق داشته باشد؟

مثلاً، خبری درباره "قرارداد چند میلیارد دلاری شرکت اپل با دولت چین برای تولید آیفون" می‌تواند همزمان به دسته‌های "فناوری"، "اقتصاد" و "سیاست" تعلق داشته باشد. این یک **مسئله طبقه‌بندی هر-از-چند** است.

حالا تصور کنید وظیفه شما تشخیص زبان یک سند است. یک سند می‌تواند همزمان به چند زبان نوشته شده باشد؟ به ندرت. این یک **مسئله طبقه‌بندی یک-از-چند** است.

در مسائل طبقه‌بندی با بیش از دو کلاس، دو نوع اصلی وظیفه وجود دارد که بسته به ماهیت کلاس‌ها انتخاب می‌شود:

🔑 طبقه‌بندی هر-از-چند (Any-of Classification)

در طبقه‌بندی هر-از-چند (که به آن چندبرچسبی یا چندمقداری نیز گفته می‌شود)، کلاس‌ها انحصاری متقابل نیستند. یک سند می‌تواند همزمان به چندین کلاس تعلق داشته باشد، یا به هیچ‌کدام از کلاس‌ها تعلق نداشته باشد.

مثال واقعی: در سیستم توصیه فیلم آمازون پرایم، یک فیلم می‌تواند همزمان برچسب‌های "اکشن"، "علمی-تخیلی" و "ماجراجویی" را داشته باشد. تصمیم برای یک برچسب، تصمیم برای دیگر را محدود نمی‌کند.

در طبقه‌بندی هر-از-چند، برای هر کلاس یک طبقه‌بند جداگانه ساخته می‌شود. برای هر کلاس، مجموعه آموزشی شامل اسناد متعلق به آن کلاس (برچسب‌های مثبت) و مکمل آن‌ها (برچسب‌های منفی) است. هنگام طبقه‌بندی یک سند جدید، هر طبقه‌بند به طور جداگانه اعمال می‌شود و تصمیم یک طبقه‌بند تأثیری بر تصمیمات طبقه‌بندهای دیگر ندارد.

🎯 طبقه‌بندی یک-از-چند (One-of Classification)

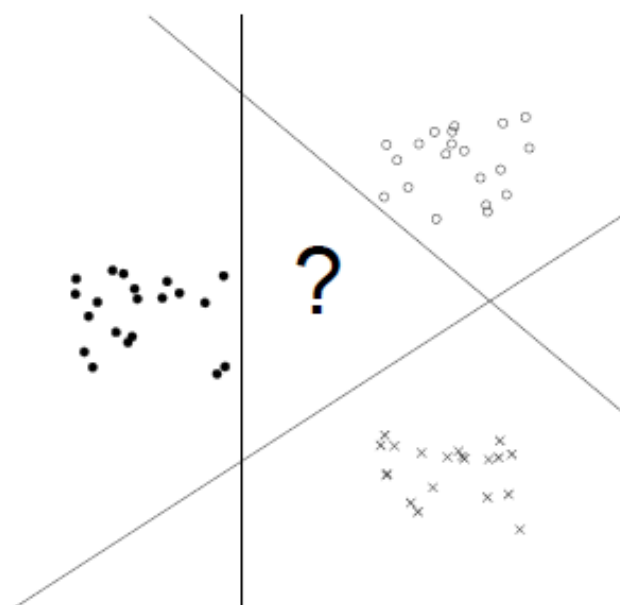
در طبقه‌بندی یک-از-چند (که به آن چندوجهی، چندکلاسی یا تک‌برچسبی نیز گفته می‌شود)، کلاس‌ها انحصاری متقابل هستند. هر سند باید دقیقاً به یکی از کلاس‌ها تعلق داشته باشد.

مثال واقعی: در سیستم تشخیص زبان گوگل ترنسلیت، یک سند باید دقیقاً به یک زبان تعلق داشته باشد: انگلیسی، فرانسوی، اسپانیایی، و غیره. در این حالت، فرض یک-از-چند یک تقریب خوب است، زیرا اکثر متون فقط به یک زبان نوشته می‌شوند.

مسائل واقعی یک-از-چند در طبقه‌بندی متن کمتر از مسائل هر-از-چند رایج هستند. با کلاس‌هایی مانند "بریتانیا"، "چین"، "طیور" یا "قهوه"، یک سند می‌تواند همزمان به چند موضوع مرتبط باشد - مانند وقتی که نخست‌وزیر بریتانیا برای صحبت درباره تجارت قهوه و طیور به چین سفر می‌کند.

📐 شکل 14.12: چالش تقسیم فضا با چند هایپرپلن

چالش اصلی در طبقه‌بندی یک-از-چند با استفاده از طبقه‌بندهای خطی این است که J هایپرپلن فضای برداری را به J ناحیه مجزا تقسیم نمی‌کنند. شکل 14.12 این مشکل را نشان می‌دهد: هایپرپلن‌ها یکدیگر را قطع می‌کنند و یک سند ممکن است همزمان به چندین ناحیه تعلق داشته باشد.



► **Figure 14.12** J hyperplanes do not divide space into J disjoint regions.

برای حل این مشکل، از روش ترکیبی استفاده می‌شود: ابتدا کلاس‌ها بر اساس امتیاز رتبه‌بندی می‌شوند و سپس کلاس برتر انتخاب می‌شود. از نظر هندسی، رتبه‌بندی می‌تواند بر اساس فاصله از J جداکننده خطی باشد. اسنادی که نزدیک به جداکننده یک کلاس هستند، احتمالاً اشتباه طبقه‌بندی می‌شوند، بنابراین هرچه فاصله از جداکننده بیشتر باشد، احتمال صحت تصمیم‌گیری مثبت بیشتر است.

📊 جدول 14.5: ماتریس سردرگمی برای تحلیل عملکرد طبقه‌بند

ماتریس سردرگمی ابزار مهمی برای تحلیل عملکرد طبقه‌بندهای چندکلاسی است. این ماتریس برای هر جفت کلاس (c_1, c_2) نشان می‌دهد که چند سند از کلاس c_1 به اشتباه به کلاس c_2 طبقه‌بندی شده‌اند.

جدول 14.5 یک ماتریس سردرگمی را برای طبقه‌بندی شش کلاس در مجموعه داده Reuters-21578 نشان می‌دهد. طبقه‌بند موفق می‌شود سه کلاس مالی (money-fx, trade, interest) را از سه کلاس کشاورزی (wheat, corn, grain) تشخیص دهد، اما در داخل هر یک از این دو گروه اشتباهات زیادی می‌کند.

	assigned class	money-fx	trade	interest	wheat	corn	grain
true class							
money-fx		95	0	10	0	0	0
trade		1	1	90	0	1	0
interest		13	0	0	0	0	0
wheat		0	0	1	34	3	7
corn		1	0	2	13	26	5
grain		0	0	2	14	5	10

► **Table 14.5** A confusion matrix for Reuters-21578. For example, 14 documents from *grain* were incorrectly assigned to *wheat*. Adapted from Picca et al. (2006).

مثال واقعی: در سیستم طبقه‌بندی ایمیل گوگل، ماتریس سردرگمی می‌تواند نشان دهد که چرا برخی ایمیل‌های "تبلیغاتی" به اشتباه به دسته "اسپم" ارسال شده‌اند. با تحلیل این ماتریس، می‌توان ویژگی‌های جدیدی معرفی کرد که این دو دسته را بهتر از هم تمایز دهد.

ماتریس سردرگمی می‌تواند به شناسایی فرصت‌هایی برای بهبود دقت سیستم کمک کند. برای مثال، برای رسیدگی به دومین خطای بزرگ در جدول 14.5 (14 در سطر grain)، می‌توان تلاش کرد ویژگی‌هایی را معرفی کند که اسناد wheat را از اسناد grain متمایز کند.

در نهایت، انتخاب بین طبقه‌بندی هر-از-چند و یک-از-چند به ماهیت مسئله بستگی دارد. اگر کلاس‌ها انحصاری متقابل هستند، طبقه‌بندی یک-از-چند می‌تواند با اعمال یک محدودیت، دقت را افزایش دهد. اما اگر کلاس‌ها می‌توانند همپوشانی داشته باشند، طبقه‌بندی هر-از-چند انتخاب مناسب‌تری است.

```
In [12]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# مرحله ۱: تولید داده‌های مصنوعی برای ۳ کلاس
# هر کلاس در یک منطقه خاص از فضا متمرکز شده است تا مسئله طبقه‌بندی قابل مشاهده باشد
def generate_multi_class_data():
    """
    داده‌هایی برای سه کلاس تولید می‌کند.
    کلاس 0: بالا-چپ (مثلاً "ورزشی")
    کلاس 1: بالا-راست (مثلاً "سیاسی")
    کلاس 2: پایین (مثلاً "فناوری")
    """
    np.random.seed(42)
    # کلاس 0 (ورزشی)
    X0 = np.random.randn(50, 2) + np.array([-2, 2])
    y0 = np.zeros(50)

    # کلاس 1 (سیاسی)
    X1 = np.random.randn(50, 2) + np.array([2, 2])
    y1 = np.ones(50)

    # کلاس 2 (فناوری)
    X2 = np.random.randn(50, 2) + np.array([0, -2])
    y2 = np.full(50, 2)

    X = np.vstack([X0, X1, X2])
    y = np.hstack([y0, y1, y2])
    return X, y

# مرحله ۲: پیاده‌سازی یک طبقه‌بند خطی ساده (از صفر)
# این طبقه‌بند فقط دو کلاس را از هم جدا می‌کند (یکی در مقابل بقیه)
class SimpleLinearClassifier:
    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.learning_rate = learning_rate
```

```

self.n_iterations = n_iterations
self.weights = None
self.bias = None

def fit(self, X, y_binary):
    """
    مدل را با داده‌های آموزشی آموزش می‌دهد.
    y_binary باید برچسب‌های 0 و 1 باشد
    """

    n_samples, n_features = X.shape
    # مقداردهی اولیه وزن‌ها و بایاس
    self.weights = np.zeros(n_features)
    self.bias = 0

    # الگوریتم بهینه‌سازی ساده (گرادیان کاهشی)
    for _ in range(self.n_iterations):
        # مدل خطی:  $z = w \cdot x + b$ 
        linear_model = np.dot(X, self.weights) + self.bias
        # تابع فعال‌سازی ساده (پله‌ای)
        y_predicted = self._step_function(linear_model)

        # به‌روزرسانی وزن‌ها و بایاس
        dw = (1 / n_samples) * np.dot(X.T, (y_predicted - y_binary))
        db = (1 / n_samples) * np.sum(y_predicted - y_binary)
        self.weights -= self.learning_rate * dw
        self.bias -= self.learning_rate * db

def predict(self, X):
    """
    برچسب کلاس (0 یا 1) را برای داده‌های جدید پیش‌بینی می‌کند.
    """
    linear_model = np.dot(X, self.weights) + self.bias
    y_predicted = self._step_function(linear_model)
    return y_predicted

def decision_function(self, X):
    """
    امتیاز (فاصله از مرز تصمیم) را برای هر نقطه محاسبه می‌کند.
    این امتیاز برای مقایسه بین طبقه‌بندها در استراتژی یک-در-برابر-بقیه حیاتی است.
    """
    return np.dot(X, self.weights) + self.bias

def _step_function(self, x):
    """
    تابع پله‌ای ساده: اگر ورودی بزرگتر از 0 بود، 1 برمی‌گرداند، در غیر این صورت 0.
    """
    return np.where(x >= 0, 1, 0)

# مرحله ۳: ساخت طبقه‌بند چندکلاسی با استراتژی "یک-در-برابر-بقیه"
class OneVsRestClassifier:
    def __init__(self):
        self.classifiers = {}
        self.classes = None

    def fit(self, X, y):
        """
        برای هر کلاس، یک طبقه‌بند دوتایی آموزش می‌دهد.
        این پیادسازی استراتژی "یک-از-چند" است که در مارکدون توضیح داده شد.
        """
        self.classes = np.unique(y)
        for cls in self.classes:
            print(f"Training classifier for class {int(cls)} vs rest...")
            # تبدیل برچسب‌ها به حالت دوتایی: 1 برای کلاس فعلی، 0 برای بقیه
            y_binary = np.where(y == cls, 1, 0)
            classifier = SimpleLinearClassifier()
            classifier.fit(X, y_binary)
            self.classifiers[cls] = classifier

    def predict(self, X):
        """
        برای هر نقطه داده، امتیاز آن را از تمام طبقه‌بندها دریافت کرده
        و کلاسی را انتخاب می‌کند که بالاترین امتیاز را داشته باشد.
        این روش رتبه‌بندی و انتخاب برتر، رامحل مشکل تقسیم فضا توسط چند هاپرپلن است.
        """
        predictions = []
        for x in X:
            scores = {}
            for cls, classifier in self.classifiers.items():
                scores[cls] = classifier.decision_function(x)

            # کلاس با بالاترین امتیاز انتخاب می‌شود
            predicted_class = max(scores, key=scores.get)
            predictions.append(predicted_class)
        return np.array(predictions)

# مرحله ۴: اجرای کل فرآیند و نمایش نتایج

# تولید داده‌ها
X, y = generate_multi_class_data()

# آموزش طبقه‌بند چندکلاسی
ovr_classifier = OneVsRestClassifier()
ovr_classifier.fit(X, y)

# پیش‌بینی روی کل مجموعه داده برای تحلیل
y_pred = ovr_classifier.predict(X)

```

```
# مرحله ۵: محاسبه و نمایش ماتریس سردرگمی (از صفر)
def calculate_confusion_matrix(y_true, y_pred, classes):
    n_classes = len(classes)
    matrix = np.zeros((n_classes, n_classes), dtype=int)
    # ایجاد نگاشت از کلاس به اندیس ماتریس
    class_to_index = {cls: i for i, cls in enumerate(classes)}

    for true_label, pred_label in zip(y_true, y_pred):
        i = class_to_index[true_label]
        j = class_to_index[pred_label]
        matrix[i, j] += 1
    return matrix

conf_matrix = calculate_confusion_matrix(y, y_pred, ovr_classifier.classes)

# چاپ نتایج
print("\n--- Confusion Matrix ---")
print("Rows: Actual Class, Columns: Predicted Class")
print(conf_matrix)

# نمایش ماتریس سردرگمی با نمودار حرارتی
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=[f"Pred {int(c)}" for c in ovr_classifier.classes],
            yticklabels=[f"Actual {int(c)}" for c in ovr_classifier.classes])
plt.title("Confusion Matrix (from scratch)")
plt.ylabel("Actual Class")
plt.xlabel("Predicted Class")
plt.show()

# نمایش مرزهای تصمیم
def plot_decision_boundaries_ovr(X, y, classifier):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                        np.arange(y_min, y_max, 0.1))

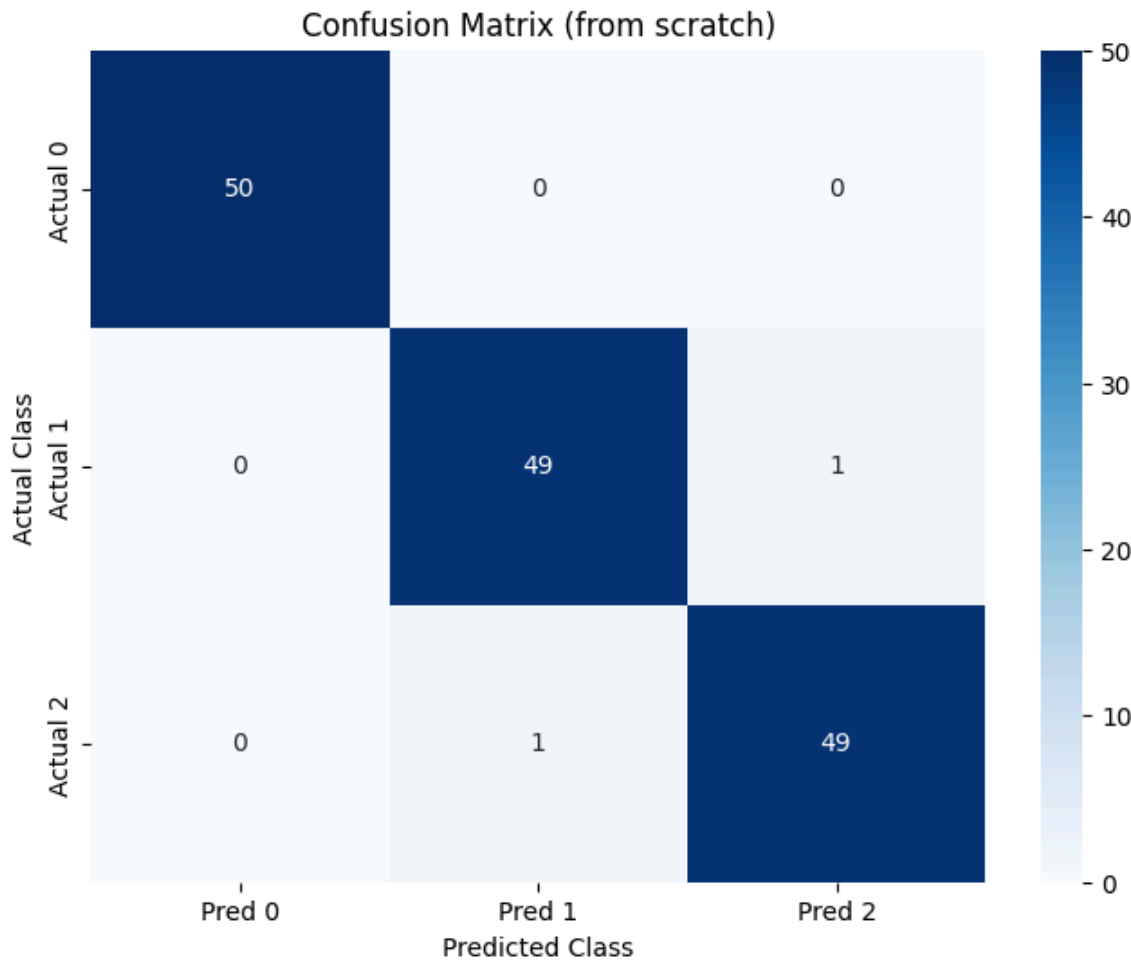
    # پیش‌بینی برای تمام نقاط در شبکه
    Z = classifier.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.4, cmap=plt.cm.Paired)
    scatter = plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k')
    plt.title("Decision Boundaries (One-vs-Rest from scratch)")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    # ایجاد لیچند (راهنما) برای کلاس‌ها
    legend1 = plt.legend(*scatter.legend_elements(),
                        title="Classes")
    plt.gca().add_artist(legend1)
    plt.show()

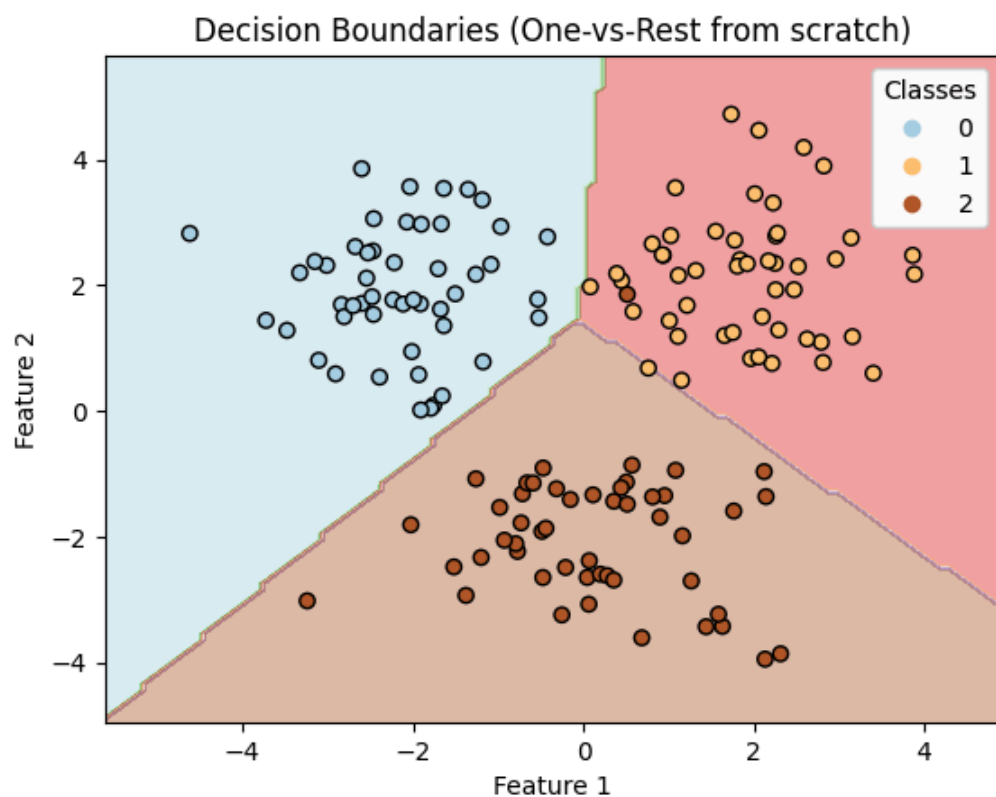
print("\n--- Plotting Decision Boundaries ---")
plot_decision_boundaries_ovr(X, y, ovr_classifier)
```

Training classifier for class 0 vs rest...
Training classifier for class 1 vs rest...
Training classifier for class 2 vs rest...

--- Confusion Matrix ---
Rows: Actual Class, Columns: Predicted Class
[[50 0 0]
 [0 49 1]
 [0 1 49]]



--- Plotting Decision Boundaries ---



14.6 مبادله بایاس-واریانس: چرا هیچ الگوریتمی کامل نیست؟ 🏴󠁧󠁢󠁥󠁮󠁧󠁿

🏴󠁧󠁢󠁥󠁮󠁧󠁿 چالش تشخیص پزشکی: آیا یک مدل ساده بهتر است یا یک مدل پیچیده؟

تصور کنید در حال ساخت یک سیستم هوشمند برای تشخیص یک بیماری نادر بر اساس علائم بیماران هستید. دو رویکرد مختلف وجود دارد:

- **مدل ساده (بایاس بالا، واریانس پایین):** یک مدل بسیار ساده که می‌گوید "هیچ‌کس بیمار نیست" چون بیماری نادر است. این مدل واریانس پایینی دارد (پیش‌بینی‌اش هرگز تغییر نمی‌کند) اما بایاس بسیار بالایی دارد (برای بیماران واقعی به طور مداوم اشتباه می‌کند). این مدل به دلیل سادگی بیش از حد، ناتوان است در الگوهای واقعی بیماری را تشخیص دهد.
- **مدل پیچیده (بایاس پایین، واریانس بالا):** یک مدل بسیار پیچیده که داده‌های آموزشی را به طور کامل به خاطر می‌سپارد، حتی شامل خطاهای اندازه‌گیری. این مدل بایاس پایینی روی داده‌های آموزشی دارد اما واریانس بالایی است. وقتی بیمار جدیدی با علائم کمی متفاوت وارد می‌شود، پیش‌بینی ممکن است به شدت اشتباه باشد. این مدل به دلیل پیچیدگی بیش از حد، به نوبه در داده‌های آموزشی حساس است.

هدف در یادگیری ماشین، یافتن تعادل بین این دو رویکرد است: مدلی که به اندازه کافی پیچیده است تا الگوهای واقعی را تشخیص دهد، اما آنقدر پیچیده نیست که توسط نویز در داده‌های آموزشی گمراه شود.

در یادگیری ماشین، مفهوم **مبادله بایاس-واریانس** نشان می‌دهد که هیچ روش یادگیری به طور جهانی بهینه نیست. این مبادله توضیح می‌دهد که افزایش پیچیدگی یک طبقه‌بند معمولاً باعث کاهش بایاس (افزایش دقت) و افزایش واریانس (کاهش پایداری) می‌شود و برعکس.

🏴󠁧󠁢󠁥󠁮󠁧󠁿 اندازه‌گیری خطا: خطای میانگین مربع (MSE)

در طبقه‌بندی، هدف ما بهینه‌سازی خطای طبقه‌بندی بر روی داده‌های جدید است، نه صرفاً به حداقل رساندن خطا روی داده‌های آموزشی. خطای طبقه‌بندی را با معیار **خطای میانگین مربع (MSE)** تعریف می‌کنیم:

$$\text{MSE}(\gamma) = \mathbb{E}_{\mathbf{d}}[\gamma(\mathbf{d}) - P(c|\mathbf{d})]^2$$

که در آن γ یک طبقه‌بند است، d یک سند، و $P(c|d)$ احتمال تعلق سند d به کلاس c است. MSE میانگین خطای طبقه‌بند را بر روی تمام اسناد اندازه‌گیری می‌کند و هم خطای ناشی از بایاس و هم خطای ناشی از واریانس را در نظر می‌گیرد.

🎯 بایاس و واریانس در طبقه‌بندهای خطی و غیرخطی

مبادله بایاس-واریانس را می‌توان با مقایسه طبقه‌بندهای خطی و غیرخطی بهتر درک کرد:

- **طبقه‌بندهای خطی (مانند Naive Bayes و Rocchio):** این طبقه‌بندها **بایاس بالایی** دارند زیرا فقط می‌توانند یک نوع مرز کلاس (هایپرپلن) را مدل کنند. اگر مرز واقعی بین کلاس‌ها پیچیده باشد (مانند شکل 14.11)، این طبقه‌بندها به طور مداوم اشتباه طبقه‌بندی می‌کنند. با این حال، این طبقه‌بندها **واریانس پایینی** دارند زیرا مرز تصمیم آن‌ها پایدار است و با تغییرات جزئی در مجموعه آموزشی، به شدت تغییر نمی‌کند.
- **طبقه‌بندهای غیرخطی (مانند kNN):** این طبقه‌بندها **بایاس پایینی** دارند زیرا می‌توانند مرزهای تصمیم پیچیده‌ای ایجاد کنند که داده‌های آموزشی را به خوبی مدل می‌کنند. با این حال، این طبقه‌بندها **واریانس بالایی** دارند زیرا به اسناد نویزی حساس هستند. یک سند نویزی در مجموعه آموزشی می‌تواند مرز تصمیم را در ناحیه اطراف خود به شدت تغییر دهد.

مثال واقعی: در سیستم فیلتر اسپم جیمیل، یک طبقه‌بند خطی ممکن است به طور مداوم ایمیل‌های تبلیغاتی خاصی را اشتباه طبقه‌بندی کند (بایاس بالا)، اما پیش‌بینی‌های آن برای کاربران مختلف پایدار است (واریانس پایین). در مقابل، یک طبقه‌بند غیرخطی ممکن است بتواند اسپم‌های جدید و پیچیده‌تر را بهتر تشخیص دهد، اما ممکن است برای کاربران مختلف، رفتار متفاوتی از خود نشان دهد (واریانس بالا).

😓 چرا بسیاری از الگوریتم‌های برتر طبقه‌بندی متن، خطی هستند؟

شاید تعجب‌آور باشد که بسیاری از بهترین الگوریتم‌های طبقه‌بندی متن، خطی هستند. مبادله بایاس-واریانس به موفقیت آن‌ها کمک می‌کند:

- در طبقه‌بندی متن، معمولاً با ابعاد بسیار بالایی سروکار داریم (هزاران یا میلیون‌ها ویژگی، یکی برای هر کلمه).
- در فضاهای با ابعاد بالا، احتمال جداپذیری خطی به سرعت افزایش می‌یابد. این یعنی یک هایپرپلن ساده اغلب می‌تواند کلاس‌ها را به خوبی از هم جدا کند.
- بنابراین، "بایاس بالا" مدل‌های خطی در عمل در مسائل طبقه‌بندی متن مشکلی جدی ایجاد نمی‌کند.
- مزیت بزرگ مدل‌های خطی، "واریانس پایین" آن‌هاست که باعث می‌شود در برابر نویز مقاوم‌تر باشند و کمتر دچار بیش‌برازش (overfitting) شوند.

مثال واقعی: در سیستم طبقه‌بندی اسناد حقوقی شرکت‌هایی مانند Google یا Microsoft، با وجود پیچیدگی ظاهری زبان حقوقی، یک طبقه‌بند خطی ساده اغلب بهترین عملکرد را دارد. زیرا با هزاران ویژگی (کلمات)، یک مرز

خطی می‌تواند اسناد مربوط به قراردادهای مختلف را به خوبی از هم جدا کند، در حالی که به نويز و تغییرات جزئی در زبان حساس نیست.

در نهایت، انتخاب بین یک روش یادگیری ساده (بایاس بالا، واریانس پایین) و یک روش پیچیده (بایاس پایین، واریانس بالا) به ماهیت مسئله، حجم داده‌ها و سطح نویز بستگی دارد. مبادله بایاس-واریانس به ما کمک می‌کند تا این تصمیم مهم را آگاهانه بگیریم و بهترین روش را برای مسئله خاص خود انتخاب کنیم.

```
# نمایش مبادله بایاس-واریانس در طبقه‌بندی #

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def generate_synthetic_data(n_samples=1000, noise_level=0.2):
    """
    تولید داده‌های مصنوعی برای نمایش مبادله بایاس-واریانس
    """

    np.random.seed(42)

    # ایجاد دو کلاس با توزیع‌های متفاوت
    X_class1 = np.random.randn(n_samples//2, 2) + np.array([2, 2])
    X_class2 = np.random.randn(n_samples//2, 2) + np.array([-2, -2])

    y = np.hstack([np.ones(n_samples//2), np.zeros(n_samples//2)])

    # ترکیب داده‌ها
    X = np.vstack([X_class1, X_class2])

    # افزودن نویز به درصدی از داده‌ها
    n_noise = int(noise_level * n_samples)
    noise_indices = np.random.choice(len(X), n_noise, replace=False)

    # معکوس کردن برچسب برخی از نقاط نویز
    y[noise_indices] = 1 - y[noise_indices]

    return X, y

def fit_classifiers(X_train, y_train):
    """
    آموزش چندین طبقه‌بند با پیچیدگی‌های متفاوت
    """

    # طبقه‌بند خطی (زیاد بایاس، کم واریانس)
    linear = LogisticRegression(max_iter=1000)
    linear.fit(X_train, y_train)

    # طبقه‌بند kNN (کم بایاس، زیاد واریانس)
    knn = KNeighborsClassifier(n_neighbors=5)
    knn.fit(X_train, y_train)

    # طبقه‌بند SVM (کم بایاس، زیاد واریانس)
    svm = SVC(kernel='rbf', C=1.0, gamma='scale')
    svm.fit(X_train, y_train)

    return {'linear': linear, 'knn': knn, 'svm': svm}

def evaluate_classifiers(classifiers, X_test, y_test):
    """
    ارزیابی طبقه‌بندها با محاسبه دقت
    """

    results = {}

    for name, clf in classifiers.items():
        y_pred = clf.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)
        results[name] = accuracy

    return results

def plot_decision_boundary(classifier, X, y, title, ax=None):
    """
    نمایش مرز تصمیم برای یک طبقه‌بند
    """

    if ax is None:
        ax = plt.gca()

    # ایجاد شبکه برای نمایش مرز
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                          np.arange(y_min, y_max, 0.1))

    # محاسبه پیش‌بینی برای هر نقطه در شبکه
    Z = classifier.predict(np.vstack([xx.ravel(), yy.ravel()]))
    Z = Z.reshape(xx.shape)

    # نمایش مرز تصمیم
    plt.contourf(xx, yy, Z, alpha=0.8)

    # نمایش داده‌ها
    plt.scatter(X[:, 0], X[:, 1], c=y, s=50, edgecolor='k', marker='o')

    # افزودن عنوان و محورها
    if title:
        plt.title(title)
    plt.xlabel('X1')
    plt.ylabel('X2')
```

```
# پیش‌بینی تمام نقاط شبکه
Z = classifier.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# نمایش مرز تصمیم و مناطق
ax.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Paired)
ax.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k')

ax.set_title(title)
ax.set_xlabel('Feature 1')
ax.set_ylabel('Feature 2')

return ax

def calculate_bias_variance(classifiers, X_train, y_train):
    """
    محاسبه بایاس و واریانس برای طبقه‌بندها
    """
    results = {}

    for name, clf in classifiers.items():
        # استفاده از اعتبارسنجی متقابل برای تخمین بایاس و واریانس
        from sklearn.model_selection import cross_val_score

        # محاسبه بایاس به عنوان 1 - میانگین دقت در اعتبارسنجی متقابل
        scores = cross_val_score(clf, X_train, y_train, cv=5)
        bias = 1 - np.mean(scores)

        # محاسبه واریانس به عنوان واریانس دقت در اعتبارسنجی متقابل
        variance = np.var(scores)

        results[name] = {'bias': bias, 'variance': variance}

    return results

# مثال کاربردی
print("Generating synthetic data to show bias-variance tradeoff:")
print("=*60)

# تولید داده‌های آموزشی و آزمونی
X, y = generate_synthetic_data(n_samples=1000, noise_level=0.2)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# آموزش طبقه‌بندها
classifiers = fit_classifiers(X_train, y_train)

# محاسبه بایاس و واریانس
bias_variance = calculate_bias_variance(classifiers, X_train, y_train)
print("\nBias and Variance of classifiers:")
for name, metrics in bias_variance.items():
    print(f"{name}: Bias={metrics['bias']:.4f}, Variance={metrics['variance']:.4f}")

# ارزیابی روی داده‌های آزمون
results = evaluate_classifiers(classifiers, X_test, y_test)
print("\nAccuracy of classifiers on test data:")
for name, accuracy in results.items():
    print(f"{name}: {accuracy:.4f}")

# نمایش مرزهای تصمیم
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

plot_decision_boundary(classifiers['linear'], X_test, y_test, 'Linear Classifier (High Bias, Low Variance)', axes[0])
plot_decision_boundary(classifiers['knn'], X_test, y_test, 'kNN Classifier (Low Bias, High Variance)', axes[1])
plot_decision_boundary(classifiers['svm'], X_test, y_test, 'SVM Classifier (Low Bias, High Variance)', axes[2])

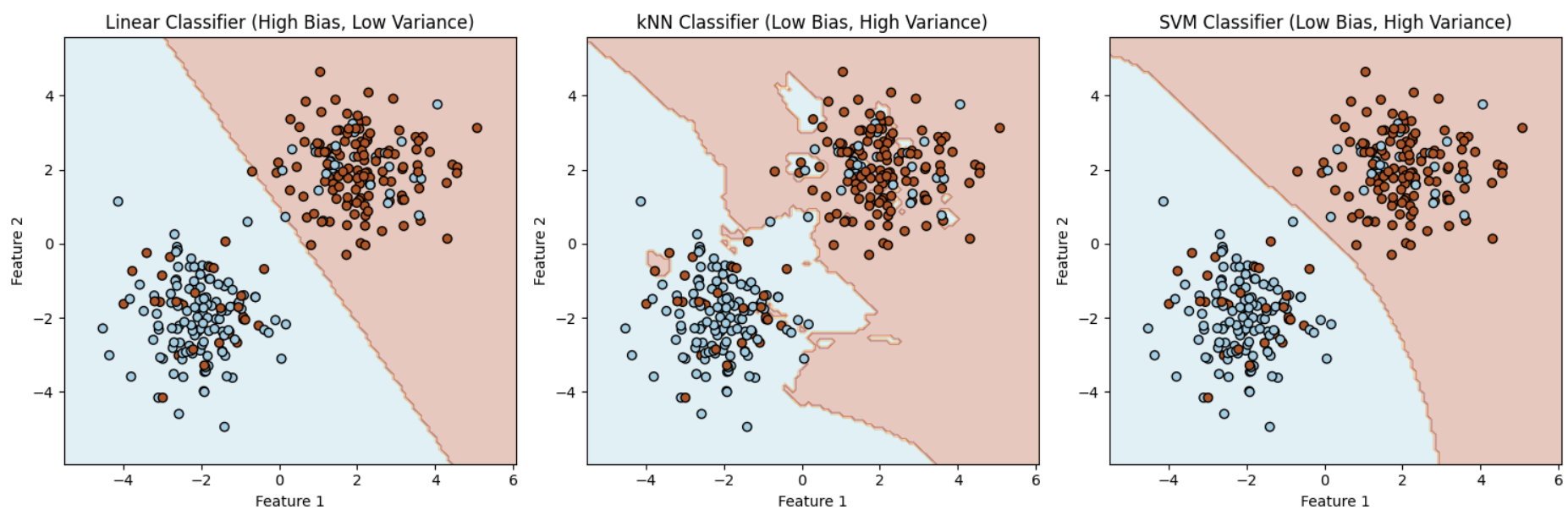
plt.tight_layout()
plt.show()

# Fixed: تحلیل اصلاح‌شده بر اساس نتایج واقعی
print("\nتحلیل:")
print("-طبقه‌بند خطی: بایاس بالا اما واریانس کم، دقت قابل قبول -")
print("-بایاس و واریانس متوسط، دقت کمی پایین‌تر: kNN طبقه‌بند -")
print("-بایاس و واریانس متوسط، بالاترین دقت: SVM طبقه‌بند -")
print("-با وجود پیچیدگی بیشتر، بهترین عملکرد را در این مجموعه داده دارد SVM این مثال نشان می‌دهد که طبقه‌بند")
```

Generating synthetic data to show bias-variance tradeoff:
=====

Bias and Variance of classifiers:
linear: Bias=0.2071, Variance=0.0002
knn: Bias=0.2543, Variance=0.0004
svm: Bias=0.2043, Variance=0.0002

Accuracy of classifiers on test data:
linear: 0.8033
knn: 0.7800
svm: 0.8067



تحلیل:

- طبقه‌بند خطی: بایاس بالا اما واریانس کم، دقت قابل قبول
- بایاس و واریانس متوسط، دقت کمی پایین‌تر: kNN طبقه‌بند
- بایاس و واریانس متوسط، بالاترین دقت: SVM طبقه‌بند

با وجود پیچیدگی بیشتر، بهترین عملکرد را در این مجموعه داده دارد SVM این مثال نشان می‌دهد که طبقه‌بند

جمع‌بندی فصل ۱۴: طبقه‌بندی فضای برداری

چگونه می‌توانیم اسناد را بر اساس شباهتشان در یک فضای چندبعدی دسته‌بندی کنیم؟ فصل ۱۴ ما را به دنیای **طبقه‌بندی فضای برداری** می‌برد؛ رویکردی که اسناد را به صورت بردارهایی در یک فضای چندبعدی نمایش می‌دهد و بر اساس فاصله یا شباهتشان تصمیم‌گیری می‌کند. این فصل نشان می‌دهد که چگونه می‌توان با استفاده از مفاهیم هندسی و جبری، مرزهای تصمیم‌گیری را بین کلاس‌های مختلف ترسیم کرد و اسناد جدید را به درستی طبقه‌بندی نمود.

- فرض **تداوم (Contiguity Hypothesis)** را به عنوان اساس طبقه‌بندی فضای برداری معرفی کردیم. این فرض می‌گوید اسناد مربوط به یک کلاس، ناحیه‌های پیوسته‌ای را در فضای برداری تشکیل می‌دهند که با ناحیه‌های کلاس‌های دیگر همپوشانی ندارند.
- الگوریتم **روچیو (Rocchio)** را به عنوان روشی برای طبقه‌بندی بر اساس **مراکز (Centroids)** بررسی کردیم. در این روش، هر کلاس با مرکز جرمی اسنادش نمایش داده می‌شود و مرزهای تصمیم‌گیری به صورت ابرصفحه‌هایی بین این مراکز ترسیم می‌گردند.
- الگوریتم **k** نزدیک‌ترین همسایه (**kNN**) را به عنوان روشی برای طبقه‌بندی محلی بررسی کردیم. در این روش، هر سند جدید بر اساس اکثریت کلاس **k** همسایه نزدیک‌ترش طبقه‌بندی می‌شود و مرزهای تصمیم‌گیری به صورت قطعه‌هایی از **تسلیم‌بندی ورونویی (Voronoi Tessellation)** شکل می‌گیرند.
- تفاوت بین **طبقه‌بندهای خطی و غیرخطی** را بررسی کردیم. طبقه‌بندهای خطی مانند روچیو و بیز ساده، مرزهای تصمیم‌گیری خطی (ابرصفحه‌ها) ایجاد می‌کنند، در حالی که روش‌هایی مانند kNN می‌توانند مرزهای تصمیم‌گیری پیچیده‌تر و غیرخطی ایجاد کنند.
- روش‌های طبقه‌بندی برای مسائل با **بیش از دو کلاس** را بررسی کردیم: مسائل **یکی-از (One-of)** که در آن هر سند باید دقیقاً به یک کلاس تعلق گیرد، و مسائل **هر-از (Any-of)** که در آن یک سند می‌تواند به چندین کلاس تعلق گیرد.
- مفهوم **مبادله-واریانس (Bias-Variance Tradeoff)** را به عنوان یکی از بنیادی‌ترین مفاهیم در یادگیری ماشین معرفی کردیم. این مفهوم توضیح می‌دهد که چرا هیچ روش طبقه‌بندی به طور جهانی بر دیگری برتری ندارد و چگونه باید بین پیچیدگی مدل و حساسیت به داده‌های آموزشی تعادل برقرار کرد.

بیا باید این مفاهیم را با چند مثال واقعی و کاربردی زنده کنیم:

♦ **دسته‌بندی اخبار در خبرگزاری‌ها:** تصور کنید در یک خبرگزاری بزرگ مانند Reuters، می‌خواهیم اخبار را به دسته‌هایی مانند «سیاسی»، «ورزشی»، «اقتصادی» و «فناوری» تقسیم کنیم. با استفاده از مدل فضای برداری، هر خبر به صورت یک بردار با وزن‌های tf-idf نمایش داده می‌شود. اخبار مربوط به «سیاسی» معمولاً کلماتی مانند «پارلمان»، «دیپلماسی»، «انتخابات» را دارند، در حالی که اخبار «ورزشی» حاوی کلماتی مانند «فوتبال»، «تیم»، «بازی» هستند. این اسناد در فضای برداری خوشه‌های مجزایی را تشکیل می‌دهند که می‌توان با ابرصفحه‌هایی بین آن‌ها مرزهای تصمیم‌گیری ترسیم کرد.

♦ **فیلتر کردن نظرات مشتریان در سایت‌های تجارت الکترونیک:** سایت‌هایی مانند Amazon از طبقه‌بندی فضای برداری برای تحلیل نظرات مشتریان استفاده می‌کنند. با استفاده از الگوریتم روچو، می‌توان مرکز هر دسته از نظرات (مثبت، منفی، خنثی) را محاسبه کرد. برای مثال، نظرات مثبت معمولاً حاوی کلماتی مانند «عالی»، «پیشنهاد می‌کنم»، «کیفیت بالا» هستند، در حالی که نظرات منفی حاوی کلماتی مانند «بد»، «نامناسب»، «خراب» هستند. وقتی نظر جدیدی ثبت می‌شود، سیستم فاصله آن را از مراکز محاسبه شده محاسبه کرده و به نزدیک‌ترین کلاس اختصاص می‌دهد.

♦ **تشخیص زبان سند با استفاده از kNN:** در یک سازمان بین‌المللی مانند اتحادیه اروپا، هزاران سند به زبان‌های مختلف دریافت می‌شود. برای تشخیص خودکار زبان هر سند، می‌توان از الگوریتم kNN استفاده کرد. هر سند به صورت یک بردار از فراوانی کلمات نمایش داده می‌شود. وقتی سند جدیدی می‌رسد، سیستم k همسایه نزدیک‌ترین آن را در مجموعه آموزشی پیدا کرده و زبان اکثریت آن همسایه‌ها را به عنوان زبان سند جدید تشخیص می‌دهد. این روش به خصوص برای زبان‌هایی که شباهت زیادی دارند (مانند هلندی و نروژی) بسیار مؤثر است.

♦ **دسته‌بندی چندبرچسبی در شبکه‌های اجتماعی:** در پلتفرم‌هایی مانند Instagram یا Twitter، یک پست می‌تواند به چندین برچسب (هشتگ) تعلق گیرد. این یک مسئله «هر-از» است که در آن یک سند (پست) می‌تواند به چندین کلاس (برچسب) تعلق گیرد. برای مثال، یک عکس از غذا در رستوران می‌تواند همزمان به برچسب‌های «#غذا»، «#رستوران»، «#زیبا» و «#سفر» تعلق گیرد. در این موارد، برای هر برچسب یک طبقه‌بند جداگانه آموزش داده می‌شود و در زمان تشخیص، هر طبقه‌بند به طور مستقل تصمیم‌گیری می‌کند.

♦ **مبادله-واریانس در سیستم‌های تشخیص پزشکی:** در سیستم‌های تشخیص پزشکی، باید بین پیچیدگی مدل و حساسیت به داده‌های آموزشی تعادل برقرار کرد. یک مدل با واریانس بالا (مانند kNN با k کوچک) ممکن است داده‌های آموزشی را به خوبی به خاطر بسپارد، اما در تشخیص بیماران جدید عملکرد ضعیفی داشته باشد. از سوی دیگر، یک مدل با مبادله بالا (مانند یک طبقه‌بند خطی ساده) ممکن است نتواند الگوهای پیچیده بیماری را تشخیص دهد. در عمل، پزشکان و محققان باید با توجه به مقدار داده‌های در دسترس و پیچیدگی مسئله، مدل مناسبی را انتخاب کنند که مبادله و واریانس متعادلی داشته باشد.

این فصل نشان داد که چگونه می‌توان با استفاده از نمایش فضای برداری، اسناد را به صورت مؤثری طبقه‌بندی کرد. روش‌هایی مانند روچو و kNN، با وجود تفاوت‌هایشان، هر دو در کاربردهای واقعی بسیار مفید هستند. درک مفهوم مبادله-واریانس به ما کمک می‌کند تا بدانیم چرا هیچ الگوریتمی به طور جهانی بر دیگری برتری ندارد و انتخاب روش مناسب به ویژگی‌های مسئله خاص بستگی دارد. در فصل‌های بعدی، با روش‌های پیشرفته‌تری مانند ماشین‌های بردار پشتیبان آشنا خواهیم شد که این مفاهیم را به سطح بالاتری ارتقا می‌دهند.